Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

Методические указания к лабораторным работам по курсу «Постреляционные базы данных»

Лабораторная работа №4

«Создание документной базы данных и работа с ней на примере СУБД MongoDb»

Виноградова М.В., Попова И., Пинская Н.М.

Под редакцией к.т.н. доц. Виноградовой М.В.

Москва, 2022 г.

ОГЛАВЛЕНИЕ

1. ЗАДАНИЕ....4
1.1. Цель работы....4
1.2. Средства выполнения....4
1.3. Продолжительность работы....4
1.4. Пункты задания для выполнения:....4
1.5. Содержание отчета....5
2. ТЕОРЕТИКО-ПРАКТИЧЕСКИЙ МАТЕРИАЛ....6
2.1. Документная база данных....6
2.2. Основные сведения о СУБД MongoDB....7
2.2.1. Основные термины....7
2.2.2. Типы данных для полей документов....8
2.2.3. Формат хранения данных....9
2.2.4. Особенность модели данных....9
2.2.5. Функции работы с данными....12
2.3. Установка и начало работы с MongoDB....12
2.4. Функции для работы с БД MongoDB....13
2.4.1. Переход к нужной БД или создание БД....13
2.4.2. Получение текущей БД....14
2.4.3. Вывод списка доступных БД....14
2.4.4. Удаление БД....15
2.5. Функции работы с коллекциями....15
2.5.1. Создание коллекций....16
2.5.2. Удаление коллекций....17
2.6. Создание документов....17
2.6.1. Вставка объектов в коллекцию....17
2.6.2. Сохранение (обновление, вставка) документа....18
2.7. Чтение документов из коллекции....19
2.7.1. Операторы сравнения....20
2.7.2. Логические операторы....22
2.7.3. Операторы для работы с массивами....23
2.7.4. Оператор для выполнения запросов с условием на наличие поля....25
2.7.5. Проекция вывода....26
2.7.6. Условия на поля вложенных структур....27
2.7.7. Вывод одного элемента коллекции....27
2.8. Изменение документов коллекций....28

2.8.1. Замена всего объекта....30
2.8.2. Операторы, используемые совместно с методом update()....31
2.8.3. Изменение элемента массива....33
2.8.4. Изменение нескольких документов коллекции....33
2.8.5. Удаление документов коллекции....35
2.9. Дополнительные операторы для чтения документов....36
2.9.1. Сортировка....36
2.9.2. Работа с операторами count, distinct, limit....37
2.10. Добавление в коллекцию связанных объектов....40
2.10.1. Создание ссылки на документ....40
2.10.2. Тип данных ObjectId....40
2.11. Группировка и агрегирование в MondoDB....42
2.11.1. Комбинация этапов при группировке....43
2.11.2. Агрегирование данных....44
2.11.3. Этапы агрегрирования данных....44
2.11.4. Список операций для агрегации документов....46
2.11.5. Примеры агрегации....46
2.12. Работа с индексами в MongoDB....48
2.12.1. Создание и использование индекса....48
2.12.2. Настройка индексов....49
2.12.3. Управление индексами....50
2.12.4. Удаление индекса....51
2.13. Режимы для вывода информации о выполнении запроса....52
2.14. Дополнительные возможности....52
2.14.1. Организация и виды кластеров....52
2.14.2. Репликация....55
2.14.3. Согласование реплик в БД NoSQL....66
2.14.4. Кворумы....68
2.15. Организация кластера на основе MongoDB....69
2.15.1. Настройка фрагментации для БД....73
2.15.2. Фрагментация целевой БД....78
3. КОНТРОЛЬНЫЕ ВОПРОСЫ....82
4. СПИСОК ИСТОЧНИКОВ....83

1. ЗАДАНИЕ

Лабораторная работа №4 «Работа с документной NOSQL БД на примере MongoDb» по курсу «Постреляционные базы данных».

1.1. Цель работы

- Изучить модель данных и способы работы с документными БД NoSql.
- Освоить методы создания документной БД и языки запросов к ним.
- Получить навыки работы с документной БД MongoDb.

1.2. Средства выполнения

- СУБД MongoDB
- Командная оболочка mongo.

1.3. Продолжительность работы

Время выполнения лабораторной работы 4 часа.

1.4. Пункты задания для выполнения:

1. Создать в среде MongoDb свою БД по теме, выданной преподавателем, содержащую не менее трех коллекций. Добавить в коллекции БД объекты сложной структуры, содержащие вложенные структуры и коллекции. Продемонстрировать (вывести на экран) содержимое коллекций.
2. Продемонстрировать изменение объектов БД:
    - добавление элементов объекта,
    - изменение элементов объекта,
    - удаление элементов объекта,
    - замена всего объекта,
    - удаление объекта.
3. Выполнить запросы к базе данных:
    - вывод всех элементов коллекции,
    - вывод с фильтрацией (условия И, ИЛИ),
    - проекция вывода (вывод части полей),
    - сортировка,

- удаление дубликатов,
- условия на поля вложенных структур,
- поиск по вложенным коллекциям объекта.

4. (расширенное задание) Выполнить запросы к базе данных:
    - проверка на наличие и отсутствие полей в документе,
    - ограничение на количество документов в результате,
    - с операторами сравнения,
    - работа с массивами,
    - с группировкой и агрегированием.
5. (дополнительно) Добавить в коллекции связанные объекты. Продемонстрировать переход по связи в запросе.
6. (дополнительно) Создать идекс и продемонстрировать работу с ним.

1.5. Содержание отчета

- Титульный лист;
- Цель работы;
- Задание;
- Тексты сценариев (создание объектов БД), запросов, команд и процедур в соответствии с пунктами задания.
- Результаты выполнения запросов (скриншоты);
- Вывод;
- Список используемой литературы.

2. ТЕОРЕТИКО-ПРАКТИЧЕСКИЙ МАТЕРИАЛ

2.1. Документная база данных

Документная база данных – это тип нереляционных баз данных (NoSQL) [1.], предназначенный для хранения и запроса данных в виде документов в структурированных форматах – BSON, а отображают данные в JSON.

Документы являются наборами атрибутов (ключ и соответствующее ему значение). Значения могут быть в свою очередь вложенными документами или массивами. Документы одного типа могут иметь общие атрибуты, а могут и не иметь - отсутствует жёстко заданная схема.

Документные базы данных [2.] позволяют разработчикам хранить и запрашивать данные в БД с помощью той же документной модели, которую они используют в коде приложения. Гибкий, полуструктурированный, иерархический характер документов и документных баз данных позволяет им развиваться в соответствии с потребностями приложений. Документная модель хорошо работает в таких примерах использования, как каталоги, пользовательские профили и системы управления контентом, где каждый документ уникален и изменяется со временем. Документные базы данных обеспечивают гибкость индексации, производительность выполнения стандартных запросов и аналитику наборов документов.

Документные базы данных реализуют документно-ориентированную модель данных. Документ-ориентированная модель носит такое название, потому что основным её элементом является документ.

Поддерживают различные функции работы с данными, такие как создание документа, поиск, редактирование, выборку документов и пр.

Особенности:

- база данных не предписывает определённый формат или схему;
- каждый документ может иметь свою внутреннюю структуру;
- документные БД являются хорошим выбором для быстрой разработки;
- в любой момент можно менять свойства данных, не изменяя структуру или сами данные.

Примеры СУБД [3.]: MongoDB [4.], CouchDB, Couchbase, MarkLogic, eXist, Berkeley DB.

2.2. Основные сведения о СУБД MongoDB

MongoDB [5.] – это нереляционная база данных с открытым исходным кодом (написана на C++), поддерживающая документоориентированные системы хранения объектов в формате, подобном JSON [6.]. Она использует гибкую модель данных, позволяющую хранить данные любой структуры, и имеет широкий набор возможностей, включая полную поддержку индексов, сегментирование и репликацию. В отличие от реляционных баз данных MongoDB предлагает документо-ориентированную модель данных, благодаря чему MongoDB работает быстрее, обладает лучшей масштабируемостью, ее легче использовать.

Одним из популярных стандартов обмена данными и их хранения является JSON (JavaScript Object Notation). JSON эффективно описывает сложные по структуре данные. Способ хранения данных в MongoDB в этом плане похож на JSON, хотя формально JSON не используется. Для хранения в MongoDB применяется формат, который называется BSON (БиСон) или сокращение от binary JSON.

BSON позволяет работать с данными быстрее: быстрее выполняется поиск и обработка. Хотя надо отметить, что BSON в отличие от хранения данных в формате JSON имеет небольшой недостаток: в целом данные в JSON-формате занимают меньше места, чем в формате BSON, с другой стороны, данный недостаток с лихвой окупается скоростью.

2.2.1. Основные термины

База данных – это физический контейнер для коллекций. Каждая база данных получает свой набор файлов в файловой системе. Один сервер MongoDB обычно имеет несколько баз данных.

Коллекция представляет собой группу документов MongoDB и существует в одной базе данных. Коллекции не используют схему. Документы в коллекции могут иметь разные поля. Как правило, все документы в коллекции имеют аналогичное или связанное назначение.

Документ – состоит из полей, это набор пар ключ-значение. Документы имеют динамическую схему. Динамическая схема означает, что документы в одной коллекции не обязательно должны иметь одинаковый набор полей или структуру, а общие поля в

документах коллекции могут содержать данные разных типов. Документы являются иерархичными структурами: значения полей документа могут быть в свою очередь вложенными документами или массивами.

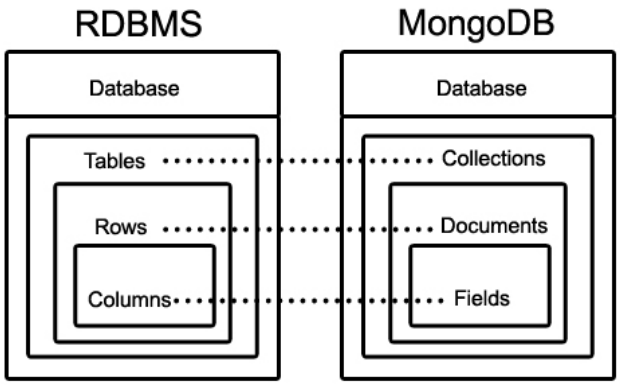


Рисунок 1. Хранение данных в РСУБД и MongoDB.

Если в традиционном SQL есть таблицы, то в MongoDB есть коллекции. И если в реляционных БД таблицы хранят однотипные жестко структурированные объекты, то в коллекции могут содержать самые разные объекты, имеющие различную структуру и различный набор свойств. Документы состоят из полей.

2.2.2. Типы данных для полей документов

MongoDB поддерживает много типов данных. Некоторые из них [7.]:

String – наиболее часто используемый тип данных для хранения данных. Строка в MongoDB должна быть действительной в формате UTF-8.

Integer – используется для хранения числового значения. Целое число может быть 32-разрядным или 64-разрядным в зависимости от вашего сервера.

Boolean – используется для хранения логического (true / false) значения.

Double – используется для хранения значений с плавающей запятой.

Клавиши Min / Max – используются для сравнения значения с самым низким и самым высоким элементами BSON.

Массивы (Arrays) – используются для хранения массивов или списка или нескольких значений в одном ключе.

Отметка времени (Timestamp) – отметка времени. Это может быть удобно для записи, когда документ был изменен или добавлен.

Объект (Object) – используется для встроенных документов.

Null – используется для хранения значения Null.

Символ (Symbol) – используется идентично строке; однако, это обычно зарезервировано для языков, которые используют определенный тип символов.

Дата (Date) – используется для хранения текущей даты или времени в формате времени UNIX. Вы можете указать собственную дату и время, создав объект Date и передав в него день, месяц, год.

Идентификатор объекта (ObjectId) – используется для хранения идентификатора документа.

Двоичные данные (Binary data) – используются для хранения двоичных данных.

Код (Code) – используется для хранения кода JavaScript в документе.

Регулярное выражение (Regular Expression) – используется для хранения регулярного выражения.

2.2.3. Формат хранения данных

Данные отображаются в формате JSON.



Рисунок 2. Пример хранения данных.

На Рисунок 2 приведен пример документа, содержащего строковые поля (name, status), числовое поле (age) и поле-масив (groups).

2.2.4. Особенность **модели данных**

MongoDB реализует документно-ориентированную модель данных.

2.2.4.1. Гибкая схема

Данные в БД MongoDB хранятся в виде коллекций. Для коллекций не задается схема. Коллекции могут содержать множество записей в виде иерархичных и полустуктурированных документов, хранящихся в формате BSON (см. Рисунок 1).

Документы могут иметь различную структуру и набор полей, а также содержать вложенные документы. Поля могут варьироваться от документа к документу; нет необходимости декларировать структуру документов в системе.

2.2.4.2. Связи между документами

Чтобы организовать связь между документами используют 2 способа:

- встраивание документов внутрь других
- ссылки.

2.2.4.3. Встроенные данные

Встроенные документы фиксируют отношения между данными, сохраняя связанные данные в единой структуре документа. Документы MongoDB позволяют встраивать структуры документа в поле или массив в документе (см. Рисунок 3). Эти денормализованные модели данных позволяют приложениям извлекать и обрабатывать связанные данные в одной операции базы данных.



Рисунок 3. Использование встроенных документов.

2.2.4.4. Ссылки

Ссылки используются для определения отношения между документами (см. Рисунок 4). Приложения могут разрешать эти ссылки для доступа к связанным данным. В целом это нормализованные модели данных.

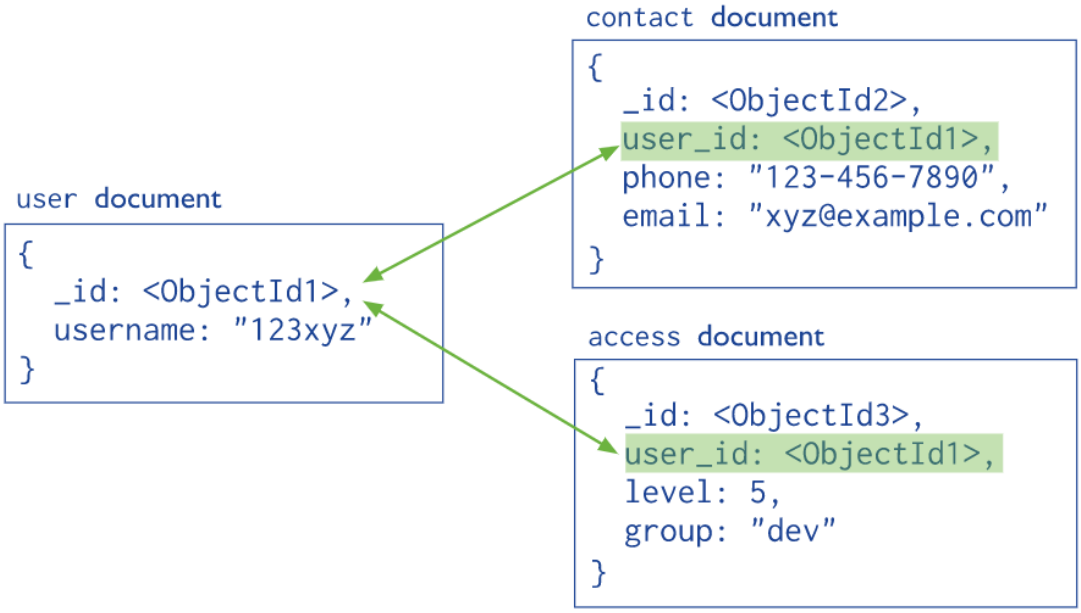


Рисунок 4. Использование ссылок.

2.2.4.5. Атомарность операций записи

В MongoDB [8.] операция записи является атомарной на уровне одного документа, даже если операция изменяет несколько встроенных документов в одном документе.

Денормализованная модель данных со встроенными данными объединяет все связанные данные в одном документе вместо нормализации по нескольким документам и коллекциям. Эта модель данных облегчает атомарные операции.

Когда одна операция записи (например, db.collection.updateMany ()) изменяет несколько документов, модификация каждого документа является атомарной, но операция в целом не является атомарной.

В ситуациях, когда требуется атомарность операций чтения и записи в нескольких документах (в одной или нескольких коллекциях), MongoDB поддерживает транзакции с несколькими документами:

- MongoDB поддерживает многодокументные транзакции на наборах реплик.
- MongoDB представляет распределенные транзакции, в которые добавлена поддержка многодокументных транзакций в сегментированных кластерах и включена существующая поддержка многодокументных транзакций в наборах реплик.

MongoDB хранит все данные в оперативной памяти для увеличения скорости работы, однако при этом является персистентной, то есть данные периодически сбрасываются на диск.

Любая СУБД подходит только для определенного класса задач, в частности MongoDB отлично подходит, если:

- Данные слабо связаны между собой (нет соединений коллекций)
- Данные не структурированы (нельзя четко выделить схему бд)
- Схема данных все время обновляется (например на старте проекта)

На рисунке (см. Рисунок 5) рассмотрена предметная область сущностей соцсети - пользователи, посты, комментарии, лайки. Как видно из рисунка каждая сущность связана с какой-либо еще. Это отрицательный пример использования MongoDB. При такой структуре предметной области хранение данных в MongoDB может только усложнить разработку.

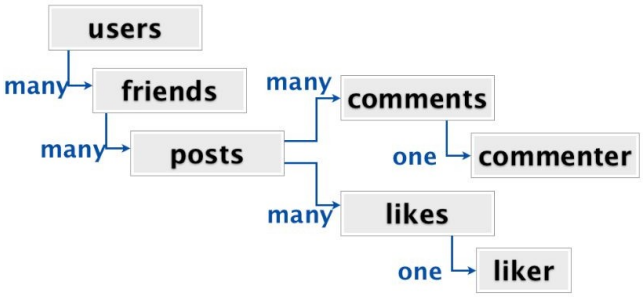


Рисунок 5. Предметная область соцсети.

2.2.5. Функции работы с данными

Базы данных MongoDB поддерживают различные функции работы с данными [9.]:

- выборка данных
- удаление данных
- изменение данных
- группировка и агрегирование
- создание индексов и работа с ними
- работа с геопространственными данными
- поиск текста и пр.

2.3. Установка и начало работы с MongoDB

Для работы с MongoDB [10.] сначала необходимо установить MongoDB на свой компьютер. [11.] Для этого посетите официальный центр загрузки (https://www.mongodb.com/download-center/community) и загрузите версию для вашей конкретной ОС.

Подробная инструкция по установке MongoDB на конкретную ОС https://docs.mongodb.com/manual/installation/. Будем использовать ОС Windows.

После установки перейдите на диск C, где вы установили MongoDB. Перейдите к программным файлам и выберите каталог MongoDB.

C: -> Program Files -> MongoDB -> Server -> 4.0(version) -> bin

В каталоге bin вы найдете пару исполняемых файлов:

- mongod
- mongo.

mongod расшифровывается как «Mongo Daemon». mongod – это фоновый процесс, используемый MongoDB. Основное назначение mongod – управлять всеми задачами сервера MongoDB. Например, принимать запросы, отвечать на запросы клиента и управлять памятью.

mongo – это оболочка командной строки, которая может взаимодействовать с клиентом (например, системными администраторами и разработчиками).

Теперь давайте посмотрим, как можно запустить сервер MongoDB. Перейдите в папку bin, которая находится в вашем каталоге MongoDB, и откройте в ней командную строку (cmd). Запустите следующую команду:

```
mongod
```

Таким образом был запущен MongoDB сервер.

Для работы с сервером потребуется клиент, поэтому внутри папки bin запустите вторую командную строку и выполните команду:

```
mongo
```

После выполнения этой команды перейдите к оболочке, которую мы запустили командой mongod (которая является сервером). В конце вы увидите сообщение «соединение принято». Это означает, что наша установка и настройка прошла успешно.

Для работы с сервером MongoDB существует множество графических интерфейсов пользователя (графический пользовательский интерфейс), таких как MongoDB Compass, Studio 3T и так далее.

Они предоставляют графический интерфейс, поэтому вы можете легко работать с базой данных и выполнять запросы, а не использовать оболочку и вводить запросы вручную.

2.4. Функции для работы с БД MongoDB

Командная оболочка mongo предоставляет ряд команд, для работы с базой данных. [12.]

2.4.1. Переход к нужной БД или создание БД

Основной синтаксис следующий:

```
use DATABASE_NAME
```

Создадим БД [13.]:

```
>use mydb
switched to db mydb
```

2.4.2. Получение текущей БД

Основной синтаксис:

```
db
```

Выведем имя текущей БД:

```
>db
mydb
```

2.4.3. Вывод списка доступных БД

Основной синтаксис следующий:

```
show dbs
```

Выведем список БД:

```
>show dbs
local     0.78125GB
test      0.23012GB
```

Созданная вами база данных (mydb) отсутствует в списке. Для отображения базы данных вам необходимо вставить в нее хотя бы один документ.

```
>db.movie.insert({"name":"Hello World!"})
>show dbs
local      0.78125GB
mydb       0.23012GB
test       0.23012GB
```

В MongoDB по умолчанию база данных является тестовой. Если вы не создали базу данных, коллекции будут храниться в тестовой базе данных.

2.4.4. Удаление БД

Основной синтаксис команды **dropDatabase ()** следующий:

```
db.dropDatabase()
```

Это удалит выбранную базу данных. Если вы не выбрали ни одну базу данных, она удалит «тестовую» базу данных по умолчанию.

Сначала проверьте список доступных баз данных с помощью команды **show dbs**.

```
>show dbs
local      0.78125GB
mydb       0.23012GB
test       0.23012GB
>
```

Если вы хотите удалить новую базу данных **<mydb>**, команда **dropDatabase()** будет выглядеть следующим образом:

```
>use mydb
switched to db mydb
>db.dropDatabase()
>{ "dropped" : "mydb", "ok" : 1 }
>
```

Теперь проверьте список баз данных.

```
>show dbs
local      0.78125GB
test       0.23012GB
>
```

2.5. Функции работы с коллекциями

Синтаксис команд, применимых к коллекции, следующий:

`<объект_бд>.<коллекция>.<функция>(<список_параметров>)`, где

<объект_бд> – Объекта базы данных («db»), в случае если команда не является глобальной;

<коллекция> – Имя коллекции;

<функция> – Имя функции;

< список_параметров > – Список параметров функции.

Список параметров задается в формате JSON.

2.5.1. Создание коллекций

Для создания коллекций в MongoDB используется метод **createCollection()** [14.].

Метод **createCollection()** имеет следующий синтаксис:

```
db.createCollection(<имя коллекции>, <свойства коллекции>)
```

Параметры метода **createCollection()** расшифровываются в Таблица 1.

Таблица 1. Параметры команды createCollection()

Параметр	Тип	Описание
Имя коллекции	String	Обязательный. Имя создаваемой коллекции.
Свойства коллекции	Document	Необязательный. Определяет такие

Параметр	Тип	Описание
		свойства, как индексация и память.

Объект, содержащий свойства коллекции, содержит следующие поля (см. Таблица 2)

Таблица 2. Поля объекта со свойствами коллекции

Свойство	Тип	Описание
capped	Boolean	Создаёт ограниченную по размеру коллекцию. Если количество элементов в коллекции достигло максимума, то каждая следующая запись будет перезаписывать более старые данные. Если вы указываете true, вам также нужно указать параметр size.
autoIndexID	Boolean	Если установлено значение true, автоматически создавать индекс для поля _id.s Значение по умолчанию – false.
size	number	(Необязательно) Указывает максимальный размер в байтах для ограниченной коллекции. Если значение capped равно true, то вам также необходимо указать это поле.
max	number	(Необязательно) Указывает максимально допустимое количество документов в коллекции.

При вставке нового документа, MongoDB, в первую очередь, проверяет параметр size, а затем – параметр max в ограниченной (capped) коллекции.

Пример:

Создадим в ней ограниченную коллекцию coaches, в которой мы сможем хранить только 10 тренеров:

```
db.createCollection("coaches", {capped: true, autoIndexID: true,
size: 371400, max: 10})
```

2.5.2. Удаление коллекций

Для удаления коллекций в MongoDB используется метод **drop()**.

Метод **drop()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.drop()
```

Пример:

Удалим коллекцию coaches с помощью метода drop().

```
db.coaches.drop()
```

2.6. Создание документов

2.6.1. Вставка объектов в коллекцию

Как и в классических реляционных СУБД insert отвечает за вставку документа в коллекцию. Вызов функции делается у коллекции. Вставляемый документ должен быть записан в JSON формате.

Метод **insert()** [15.] имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.insert(<документ> или <массив документов>)
```

Параметры метода **insert()** расшифровываются в Таблица 3.

Таблица 3. Параметры команды insert()

Параметр	Тип	Описание
document	Документ или массив документов	Обязательный. Документ или массив документов для вставки в коллекцию.

Пример:

Добавим нового тренера в БД.

Коллекция тренерского состава (coaches). Обращаемся к коллекции coaches (см. Рисунок 6).

```
db.coaches.insert({"fio": "Иванов И.И.", "birth_date": "14.02.1990",
"category":"Первая категория", "contacts":{"phone":"7977654123",
"email":"ivanov@gmail.com"}, "awards":["Лучший тренер 2019"], "sport":
"волейбол"})
```

```
> db.coaches.insert({"fio": "Иванов И.И.", "birth_date": "14.02.1990", "category":"Первая категория","contacts":{"phone"
:"7977654123","email":"ivanov@gmail.com"}, "awards":["Лучший тренер 2019"], "sport": "волейбол"})
WriteResult({ "nInserted" : 1 })
> db.coaches.insert({"fio": "Петров П.П.", "birth_date": "18.09.1989", "category":"Высшая категория","contacts":{"phone"
:"7977614520","email":"petrov@gmail.com"}, "awards":["Лучший тренер 2019", "Лучший футболист года"], "sport": "футбол"})

WriteResult({ "nInserted" : 1 })
> db.coaches.insert({"fio": "Сидоров С.С.", "birth_date": "18.09.1991", "category":"Первая категория","contacts":{"phone
":"8978614544","email":"sidorov@gmail.com"}, "awards":["3а первое место в соревновании по синхронному плаванию", "Лучший
 пловец года"], "sport": "плавание"})
WriteResult({ "nInserted" : 1 })
> db.coaches.insert({"fio": "Степанов А.А.", "birth_date": "18.09.1990", "category":"Вторая категория","contacts":{"phon
e":"8978222511","email":"stepanov@gmail.com"}, "awards":["Тренер года 2020", "Лучший легкоатлет России"], "sport": "легк
ая атлетика"})
WriteResult({ "nInserted" : 1 })
> db.coaches.insert({"fio": "Каленов А.П.", "birth_date": "03.05.1989", "category":"Первая категория","contacts":{"phone
":"7971513527","email":"kalenov@gmail.com"}, "awards":["Лучший тренер 2019", "Лучший футболист года"], "sport": "футбол"
})
WriteResult({ "nInserted" : 1 })
```

Рисунок 6. Результат работы команды insert().

2.6.2. Сохранение (обновление, вставка) документа

Метод **save()** [16.] используется для обновления существующего документа или вставки нового документиа [17.]. Метод **save()** работает таким же образом, как и метод **insert()**, в случае, если мы не указываем _id. Если же мы указываем данное поле, тогда документ перезапишет все данные документа с таким _id.

Метод **save()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.save(<документ>)
```

Параметры метода **save()** расшифровываются в Таблица 4.

Таблица 4. Параметры команды save()

Параметр	Тип	Описание
document	Документ	Обязательный. Документ для сохранения в коллекцию.

Пример:

Сохраним в БД новую группу по художественной гимнастике (см. Рисунок 7).

```
db.groups.save({title: "Группа по художественной гимнастике", type: "Подростковая", col: 24})
```

```
> db.groups.save({title: "Группа по художественной гимнастике", type: "Подростковая", col: 24})
WriteResult({ "nInserted" : 1 })
```

Рисунок 7. Результат работы команды save().

2.7. Чтение документов из коллекции

Для выборки документов из коллекции [18.] используется метод **find()** [19.]. Find — аналог SELECT в MySQL. Используется для выборки документов из MongoDB. Возвращает массив документов в виде коллекции, если документов нет — пустую коллекцию.

Метод **find()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.find(<запрос>, <проекция>)
```

Параметры метода **find()** расшифровываются в Таблица 5.

Таблица 5. Параметры команды find()

Параметр	Тип	Описание
Запрос	Документ	Необязательный. Критерий отбора с помощью формального запроса операторов. Чтобы вернуть все документы в коллекции, пропустите этот

Параметр	Тип	Описание
		параметр или передайте пустой документ ({}).
Проекция	Документ	Необязательный. Задает поля, которые будут возвращены в результате обработки запроса и поле «_id», если не указано.

При вызове функции без параметров возвратятся все документы коллекции.

Использование метода **pretty()** позволяет отобразить результаты в удобном для чтения формате.

Выведем созданные документы в коллекции coaches (см. Рисунок 8):

```
db.coaches.find().pretty()
```

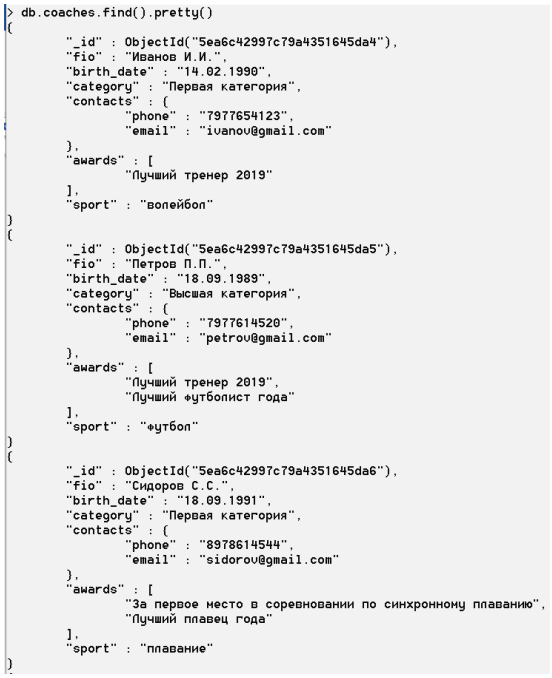


Рисунок 8. Результат работы команды find().

С методом **find()** могут использоваться ряд операторов запроса для фильтрации данных.

2.7.1. Операторы сравнения

Операторы сравнения представлены в Таблица 6.

Таблица 6. Операторы сравнения.

Оператор	Общий синтаксис	Параметры	Описание
$eq	{ <field>: { $eq: <value> } }	Скалярное значение, массив, документ	Определяет условие равенства. Сопоставляет документы, в которых значение поля равно указанному значению.
$gt	{field: {$gt: value} }	Для большинства типов данных операторы	Выбирает те документы, в которых

Оператор	Общий синтаксис	Параметры	Описание
		сравнения выполняют сравнения только с полями, в которых тип BSON соответствует типу значения запроса.	значение поля больше (т.е.>) указанного значения.
$gte	{field: {$gte: value} }	Для большинства типов данных операторы сравнения выполняют сравнение только с полями, в которых тип BSON соответствует типу значения запроса.	Выбирает документы, в которых значение поля больше или равно (то есть> =) указанному значению.
$lt	{field: {$lt: value} }	Для большинства типов данных операторы сравнения выполняют сравнения только с полями, в которых тип BSON соответствует типу значения запроса.	Выбирает документы, в которых значение поля меньше (то есть <) указанного значения.
$lte	{ field: { $lte: value} }	Для большинства типов данных операторы сравнения выполняют сравнения только с полями, в которых тип BSON соответствует типу значения запроса.	Выбирает документы, в которых значение поля меньше или равно (то есть <=) указанному значению.
$ne	{field: {$ne: value} }	Для большинства типов данных операторы сравнения выполняют сравнения только с полями, в которых тип BSON соответствует типу значения запроса.	Выбирает документы, где значение поля не равно указанному значению.

Пример:

Выведем список групп, в которых занимается 20 человек (см. Рисунок 9).

```
db.groups.find({"col": {$eq:20}}).pretty()
```

```
> db.groups.find({"col": {$eq:20}}).pretty()
{
        "_id" : ObjectId("5ea752eda42be99201356ec2"),
        "title" : "Группа по футболу 1",
        "col" : 20,
        "type" : "Подростковая",
        "pupils" : [
                "Сенин С.С.",
                "Степовой А.А",
                "Андреев К.К."
        ]
}
{
        "_id" : ObjectId("5ea752eea42be99201356ec4"),
        "title" : "Группа по атлетике 1",
        "col" : 20,
        "type" : "Подростковая",
        "pupils" : [
                "Соколов М.М.",
                "Жданов А.А.",
                "Кириллов К.К."
        ]
}
```

Рисунок 9. Результат работы команды find() с оператором $eq.

Выведем список групп, в которых занимается менее 20 человек (см. Рисунок 10).

```
db.groups.find({"col": {$lt:20}}).pretty()
```

```
> db.groups.find({"col": {$lt:20}}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645da9"),
        "title" : "Группа по плаванию 1",
        "col" : 10,
        "type" : "Детская",
        "pupils" : [
                "Иванникова И.И.",
                "Маслов А.А."
        ]
}
```

Рисунок 10. Результат работы команды find() с оператором $lt.

Выведем список групп, в которых занимается более 20 человек (см. Рисунок 11).

```
db.groups.find({"col": {$gt:20}}).pretty()
```

```
> db.groups.find({"col": {$gt:20}}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645dad"),
        "title" : "Группа по атлетике 1",
        "col" : 24,
        "pupils" : [
                "Соколов М.М.",
                "Жданов А.А.",
                "Кириллов К.К."
        ]
}
```

Рисунок 11. Результат работы команды find() с оператором $gt.

2.7.2. Логические операторы

Логические операторы представлены в Таблица 7.

Таблица 7. Логические операторы

Оператор	Общий синтаксис	Параметры	Описание
$and	{ $and: [{ <expression1> }, { <expression2> } , ... , { <expressionN> }] }	Массив документов	выполняет логическую операцию И над массивом из одного или нескольких выражений (например, <expression1>, <expression2> и т. д.) и выбирает документы, которые удовлетворяют всем выражениям в массиве.
$not	{ field: { $not: { <operator-expression> } } }	Документ, регулярное выражение	Выполняет логическую операцию NOT над указанным <оператор-выражением> и выбирает документы, которые не соответствуют <оператор-выражению>.
$nor	{ $nor: [{ <expression1> }, { <expression2> }, ... { <expressionN> }] }	Массив документов.	Выполняет логическую операцию NOR для массива из одного или нескольких выражений запроса и выбирает документы, которые не соответствуют всем выражениям запросов в массиве.
$or	{ $or: [{ <expression1> }, { <expression2> }, ... , { <expressionN> }] }	Массив документов.	Выполняет логическую операцию ИЛИ над массивом из двух или более <выражений> и выбирает документы, которые удовлетворяют хотя бы одному из <выражений>.

Пример:

Выведем всех тренеров высшей категории по футболу (см. Рисунок 12).

```
db.coaches.find({$and: [{"category":"Высшая категория"},
{"sport":"футбол"}]}).pretty()
```

```
> db.coaches.find({$and: [{"category":"Высшая категория"}, {"sport":"футбол"}]}).pretty()
{
        "_id" : ObjectId("5ea6c42997c79a4351645da5"),
        "fio" : "Петров П.П.",
        "birth_date" : "18.09.1989",
        "category" : "Высшая категория",
        "contacts" : {
                "phone" : "7977614520",
                "email" : "petrov@gmail.com"
        },
        "awards" : [
                "Лучший тренер 2019",
                "Лучший футболист года"
        ],
        "sport" : "футбол"
}
```

Рисунок 12. Результат работы команды find() с оператором $and.

Выведем всех тренеров высшей категории либо вид спорта у которых – футбол (см. Рисунок 13).

```
db.coaches.find({$or: [{"category":"Высшая категория"},
{"sport":"футбол"}]}).pretty()
```

```
> db.coaches.find({$or: [{"category":"Высшая категория"}, {"sport":"футбол"}]}).pretty()
{
        "_id" : ObjectId("5ea6c42997c79a4351645da5"),
        "fio" : "Петров П.П.",
        "birth_date" : "18.09.1989",
        "category" : "Высшая категория",
        "contacts" : {
                "phone" : "7977614520",
                "email" : "petrov@gmail.com"
        },
        "awards" : [
                "Лучший тренер 2019",
                "Лучший футболист года"
        ],
        "sport" : "футбол"
}
{
        "_id" : ObjectId("5ea6c42997c79a4351645da8"),
        "fio" : "Каленов А.П.",
        "birth_date" : "03.05.1989",
        "category" : "Первая категория",
        "contacts" : {
                "phone" : "7971513527",
                "email" : "kalenov@gmail.com"
        },
        "awards" : [
                "Лучший тренер 2019",
                "Лучший футболист года"
        ],
        "sport" : "футбол"
}
```

Рисунок 13. Результат работы команды find() с оператором $or.

2.7.3. Операторы для работы с массивами

Операторы для работы с массивами представлены в Таблица 8.

Таблица 8. Операторы для работы с массивами

Оператор	Общий синтаксис	Параметры	Описание
$all	{ <field>: { $all: [<value1>, <value2> ...] } }	Массив скалярных значений, вложенных массивов	Выбирает документы, в которых значение поля представляет собой массив, содержащий все указанные элементы.
$elemMatch	{ <field>: { $elemMatch: { <query1>, <query2>, ...} } }	Документ, содержащий запросы ($gt, $lt и др.).	Сопоставляет документы, содержащие поле массив, по крайней мере с одним элементом, который соответствует всем указанным критериям запроса.
$size	db.collection.find({ field: { $size: number } });	Целое число.	Возвращает все документы в коллекции, где поле представляет собой массив из указанного числа элементов.
$nin	{ field: { $nin: [<value1>, <value2>, ..., <valueN>]} }	Массив скалярных значений.	Выбирает документы где: значение поля не находится в указанном

Оператор	Общий синтаксис	Параметры	Описание
			массиве или поле не существует.
$in	{ field: { $in: [<value1>, <value2>, ..., <valueN>] } }	Массив скалярных значений, регулярных выражений.	Выбирает документы, в которых значение поля равно любому значению в указанном массиве.

Примеры:

Запрос выбирает все объекты, которые содержат в поле pupils учащихся Соколов М.М., Жданов А.А (см. Рисунок 14).

```
db.groups.find({"pupils": {$all: ["Соколов М.М.", "Жданов А.А."]}}).pretty()
```

```
> db.groups.find({"pupils": {$all: ["Соколов М.М.", "Жданов А.А."]}}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645dad"),
        "title" : "Группа по атлетике 1",
        "col" : 24,
        "pupils" : [
                "Соколов М.М.",
                "Жданов А.А.",
                "Кириллов К.К."
        ]
}
```

Рисунок 14. Результат работы команды find() с оператором $all.

Выберем ФИО и вид спорта тех тренеров, которые имеют указанные награды (поле массив awards) (см. Рисунок 15).

```
db.coaches.find({"awards": {$in: ["Тренер года 2020", "Лучший тренер 2019"]}}, {_id: 0, fio: 1, sport:1}).pretty()
```

```
> db.coaches.find({"awards": {$in: ["Тренер года 2020", "Лучший тренер 2019"]}}, {_id: 0, fio: 1, sport:1}).pretty()
{ "fio" : "Иванов И.И.", "sport" : "волейбол" }
{ "fio" : "Петров П.П.", "sport" : "футбол" }
{ "fio" : "Степанов А.А.", "sport" : "легкая атлетика" }
{ "fio" : "Каленов А.П.", "sport" : "футбол" }
```

Рисунок 15. Результат работы команды find() с оператором $in и проекцией.

Выберем ФИО и вид спорта тех тренеров, которые не имеют указанных наград (поле массив awards) (см. Рисунок 16).

```
db.coaches.find({"awards": {$nin: ["Тренер года 2020", "Лучший тренер 2019"]}}, {_id: 0, fio: 1, sport:1}).pretty()
```

```
> db.coaches.find({"awards": {$nin: ["Тренер года 2020", "Лучший тренер 2019"]}}, {_id: 0, fio: 1, sport:1}).pretty()
{ "fio" : "Сидоров С.С.", "sport" : "плавание" }
```

Рисунок 16. Результат работы команды find() с оператором $nin.

Выполним запрос: выводим сведения о группе, в которой поле pupils содержит учащегося с ФИО Иванников И.И. возрастом 15 лет (см. Рисунок 17).

```
db.groups.find({"pupils":{$elemMatch: {"fio":"Иванников И.И.", "date_birth":"12.12.2012"}}}).pretty()
```

```
> db.groups.find({"pupils": {$elemMatch: {"fio":"Иванников И.И.", "date_birth":"12.12.2012"}}}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645da9"),
        "title" : "Группа по плаванию 1",
        "col" : 10,
        "type" : "Детская",
        "pupils" : [
                {
                        "fio" : "Иванников И.И.",
                        "date_birth" : "12.12.2012"
                },
                {
                        "fio" : "Маслов А.А.",
                        "date_birth" : "18.03.2012"
                }
        ]
}
```

Рисунок 17. Результат работы команды find() с оператором $elemMatch.

2.7.4. Оператор для выполнения запросов с условием на наличие поля

Описание оператора для проверки наличия поля представлено в Таблица 9.

Таблица 9. Оператор для проверки наличия поля

Оператор	Общий синтаксис	Параметры	Описание
$exists	{ field: { $exists: <boolean> } }	Boolean	Когда значение равно true, $exists соответствует документам, которые содержат поле, включая документы, в которых значение поля равно нулю. Если значение равно false, запрос возвращает только те документы, которые не содержат поле.

Подробнее об операторах фильтрации можно прочитать на официальном сайте MongoDB: [20.]

Пример:

Выведем список групп, у которых установлено допустимое число мест (см. Рисунок 18).

```
db.groups.find({"col": {$exists:true}}).pretty()
```

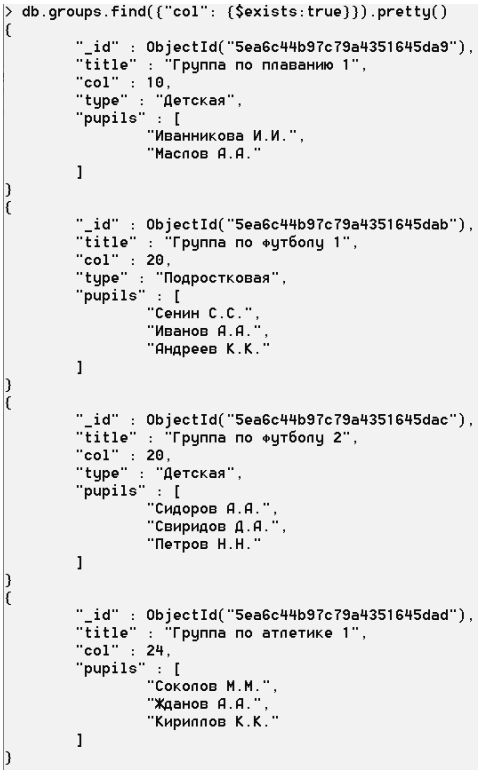


Рисунок 18. Результат работы команды find() с оператором $exists.

2.7.5. Проекция вывода

Проекция вывода [21.] имеет следующий синтаксис:

```
{ field1: <boolean>, field2: <boolean> ... }
```

Документ может иметь множество полей, но не все эти поля нам могут быть нужны и важны при запросе. И в этом случае мы можем включить в выборку только нужные поля, использовав проекцию. Например, выведем только порцию информации, например, найдем группу с названием «Группа по футболу 1», включив в результирующую выборку только список учащихся (см. Рисунок 19).

```
db.groups.find({"title":"Группа по футболу 1"}, {"_id": 0, "pupils": 1})
```

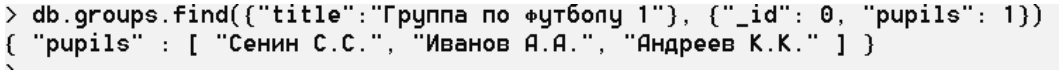


Рисунок 19. Результат работы команды find() с проекцией.

Использование единицы в качестве параметра { title: 1} указывает, что запрос должен вернуть только содержание свойства title. Нужно найти все поля документа кроме свойства _id. В этом случае в качестве параметра указываем 0. Альтернативно вместо 1 и 0 можно использовать true и false.

Если не нужно конкретизировать выборку, а следует вывести все документы, то можно оставить первые фигурные скобки пустыми. Чтобы не выводить идентификатор, поле _id указываем 0 (см. Рисунок 20).

```
db.groups.find({}, {"_id": 0, "title": 1, "pupils": 1})
```

```
> db.groups.find({}, {"_id": 0, "title": 1, "pupils": 1})
{ "title" : "Группа по плаванию 1", "pupils" : [ "Иванникова И.И.", "Маслов А.А." ] }
{ "title" : "Группа по футболу 1", "pupils" : [ "Сенин С.С.", "Иванов А.А.", "Андреев К.К." ] }
{ "title" : "Группа по футболу 2", "pupils" : [ "Сидоров А.А.", "Свиридов Д.А.", "Петров Н.Н." ] }
{ "title" : "Группа по атлетике 1", "pupils" : [ "Соколов М.М.", "Жданов А.А.", "Кириллов К.К." ] }
{ "title" : "Группа по волейболу 1", "pupils" : [ "Зайцев М.М.", "Иванов А.А.", "Гришин К.К." ] }
```

Рисунок 20. Результат работы команды find() с проекцией.

2.7.6. Условия на поля вложенных структур

Для фильтрации документов по полю структуры используется точечный синтаксис (оператор точка) «.».

Пример:

Производим поиск по полям структуры contacts. Данный запрос выводит тренера, email которого ivanov@gmail.com (см. Рисунок 21).

```
db.coaches.find({"contacts.email": "ivanov@gmail.com"}).pretty()
```

```
> db.coaches.find({"contacts.email": "ivanov@gmail.com"}).pretty()
{
        "_id" : ObjectId("5ea6c42997c79a4351645da4"),
        "fio" : "Иванов И.И.",
        "birth_date" : "14.02.1990",
        "category" : "Первая категория",
        "contacts" : {
                "phone" : "7977654123",
                "email" : "ivanov@gmail.com"
        },
        "awards" : [
                "Лучший тренер 2019"
        ],
        "sport" : "волейбол"
}
```

Рисунок 21. Результат работы команды find() с поиском по полям структуры.

2.7.7. Вывод одного элемента коллекции

Метод **findOne** [22.] работает аналогично **find**, но возвращает первый совпавший документ.

Метод **findOne()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.findOne(<запрос>, <проекция>)
```

Параметры метода **findOne()** расшифровываются в Таблица 10.

Таблица 10. Параметры команды findOne()

Параметр	Тип	Описание
Запрос	Документ	Необязательный. Критерий отбора с помощью формального запроса операторов. Чтобы вернуть документ в коллекции, пропустите этот параметр или передайте пустой документ ({}).
Проекция	Документ	Необязательный. Задает поля, которые будут возвращены в результате обработки запроса и поле «_id», если не указано.

С методом **findOne()** используются те же операторы фильтрации данных, что и с методом **find()**.

Пример:

Выведем первого тренера из коллекции coaches (см. Рисунок 22).

```
db.coaches.findOne()
```

```
> db.coaches.findOne()
{
        "_id" : ObjectId("5ea6c42997c79a4351645da4"),
        "fio" : "Иванов И.И.",
        "birth_date" : "14.02.1990",
        "category" : "Первая категория",
        "contacts" : {
                "phone" : "7977654123",
                "email" : "ivanov@gmail.com"
        },
        "awards" : [
                "Лучший тренер 2019"
        ],
        "sport" : "волейбол"
}
```

Рисунок 22. Результат работы команды findOne().

2.8. Изменение документов коллекций

Иногда требуется произвести обновление данных в документе, либо заменить весь документ. Для этого используется метод «**update**». [23.]

Метод **update()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.update(<запрос>, <данные, на которые нужно заменить>, <настройки>)
```

Параметры метода **update()** расшифровываются в Таблица 11.

Таблица 11. Параметры команды update().

Параметр	Тип	Описание
Запрос	Документ	Обязательный. Критерий (запрос), который используется для поиска требуемого документа.
Данные, на которые нужно заменить	Документ	Обязательный. Данные, используемые для замены.
Настройки		
upsert	boolean	Необязательный. Если установлено значение true, создает новый документ, когда ни один документ не соответствует критериям запроса. Значением по умолчанию является false, которое не вставляет новый документ, если совпадение не найдено.
multi	boolean	Необязательный. Если установлено значение true, обновляет несколько документов, которые соответствуют критериям запроса. Если установлено значение false, обновляется один документ. Значение по умолчанию false.
collation	Документ	Необязательный. Сопоставление позволяет пользователям задавать языковые правила для сравнения строк, например правила для букв и знаков ударения.
arrayFilters	Массив	Необязательный. Массив документов, содержащий условия фильтрации, которые определяют, какие элементы массива нужно изменить для операции обновления в поле массива.
hint	Документ или строка	Необязательный. Документ или строка, в которой указывается индекс, используемый для поддержки предиката запроса. Опция может принимать документ спецификации индекса или строку имени индекса.

Пример:

Обновим группу (коллекция groups): установим допустимое число мест 30 штук у Группа по волейболу 1 (см. Рисунок 23).

```
db.groups.update({"title":"Группа по волейболу 1"}, {$set: {"col":30}})
```

```
> db.groups.update({"title":"Группа по волейболу 1"}, {$set: {"col":30}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Рисунок 23. Результат работы команды update().

Проверим, что изменения применились (см. Рисунок 24):

```
> db.groups.find({"title": "Группа по волейболу 1"}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645dae"),
        "title" : "Группа по волейболу 1",
        "col" : 30,
        "type" : "Подростковая",
        "pupils" : [
                "Зайцев М.М.",
                "Иванов А.А.",
                "Гришин К.К."
        ]
}
```

Рисунок 24. Вывод изменений.

2.8.1. Замена всего объекта

```
db.groups.update({"title":"Группа по легкой атлетике 1"}, {"title": "Группа по атлетике 1", "col": 24, "pupils":["Соколов М.М.", "Жданов А.А.", "Кириллов К.К."]})
```

```
> db.groups.update({"title":"Группа по легкой атлетике 1"}, {"title": "Группа по атлетике 1", "col": 24, "pupils":["Соколов М.М.", "Жданов А.А.", "Кириллов К.К."]})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Рисунок 25. Результат работы команды update().

До (см. Рисунок 26):

```
> db.groups.find({"title":"Группа по легкой атлетике 1"}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645dad"),
        "title" : "Группа по легкой атлетике 1",
        "col" : 15,
        "type" : "Подростковая",
        "pupils" : [
                "Серов М.М.",
                "Жданов А.А.",
                "Авдеева К.К."
        ]
}
```

Рисунок 26. Данные до изменения.

После обновления (см. Рисунок 27):

```
> db.groups.find({"title":"Группа по атлетике 1"}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645dad"),
        "title" : "Группа по атлетике 1",
        "col" : 24,
        "pupils" : [
                "Соколов М.М.",
                "Жданов А.А.",
                "Кириллов К.К."
        ]
}
```

Рисунок 27. Данные после изменения.

2.8.2. Операторы, используемые совместно с методом update()

Операторы для метода **update()** представлены в Таблица 12. [24.]

Таблица 12. Операторы для команды update().

Оператор	Общий синтаксис	Параметры	Описание
$set	{ $set: { <field1>: <value1>, ... } }	Документ	Обновление или создание поля
$unset	{ $unset: { <field1>: "", ... } }	Документ	Удаление поля
$inc	{ $inc: { <field1>: <amount1>, <field2>: <amount2>, ... } }	Документ	Увеличение значения поля на заданное число
$pop	{ $pop: { <field>: <-1 | 1>, ... } }	Документ	Удаляет последний (или первый) элемент массива
$pushAll	{ $pullAll: { <field1>: [<value1>, <value2> ...], ... } }	Документ	Помещает несколько элементов в массив
$push	{ $push: { <field1>: <value1>, ... } }	Документ	Помещает новый элемент в массив
$addToSet	{ $addToSet: { <field1>: <value1>, ... } }	Документ	Помещает новый элемент в массив (исключаются дубликаты)
$pull	{ $pull: { <field1>: <value| condition>, <field2>: <value| condition>, ... } }	Документ	Удаляет из массива значение (при его наличии)
$pullAll	{ $pullAll: { <field1>: [<value1>, <value2> ...], ... } }	Документ	Удаляет из массива все подходящие значения

Пример:

Обновим объект группы по волейболу №1: удалим поле «число мест» (поле col) (см. Рисунок 28).

```
db.groups.update({"title": "Группа по волейболу 1"}, {$unset: {"col":""}})
```

```
> db.groups.update({"title": "Группа по волейболу 1"}, {$unset: {"col":""}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Рисунок 28. Изменение данных при помощи update() с оператором $unset.

Выполним проверку (см. Рисунок 29):

```
> db.groups.find({"title": "Группа по волейболу 1"}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645dae"),
        "title" : "Группа по волейболу 1",
        "type" : "Подростковая",
        "pupils" : [
                "Зайцев М.М.",
                "Иванов А.А.",
                "Гришин К.К."
        ]
}
```

Рисунок 29. Данные после изменения при помощи update() с оператором $unset.

Добавим в массив pupils нового учащегося Захарова А.А. (см. Рисунок 30).

```
db.groups.update({"title":"Группа по футболу 1"}, {$addToSet:
{"pupils":"Захаров А.А."}})
```

```
> db.groups.update({"title":"Группа по футболу 1"}, {$addToSet: {"pupils":"Захаров А.А."}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Рисунок 30. Изменение данных при помощи update() с оператором $ addToSet.

Проверим (см. Рисунок 31):

```
> db.groups.find({"title": "Группа по футболу 1"}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645dab"),
        "title" : "Группа по футболу 1",
        "col" : 20,
        "type" : "Подростковая",
        "pupils" : [
                "Сенин С.С.",
                "Иванов А.А.",
                "Андреев К.К.",
                "Захаров А.А."
        ]
}
```

Рисунок 31. Данные после изменения при помощи update() с оператором $addToSet.

Удалим из массива pupils учащегося Захарова А.А. (см. Рисунок 32).

```
db.groups.update({"title":"Группа по футболу 1"}, {$pull:
{"pupils":"Захаров А.А."}})
```

```
> db.groups.update({"title":"Группа по футболу 1"}, {$pull: {"pupils":"Захаров А.А."}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Рисунок 32. Изменение данных при помощи update() с оператором $pull.

Проверим (см. Рисунок 33):

```
> db.groups.find({"title": "Группа по футболу 1"}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645dab"),
        "title" : "Группа по футболу 1",
        "col" : 20,
        "type" : "Подростковая",
        "pupils" : [
                "Сенин С.С.",
                "Иванов А.А.",
                "Андреев К.К."
        ]
}
```

Рисунок 33. Данные после изменения при помощи update() с оператором $pull.

2.8.3. Изменение элемента массива

Оператор $set заменяет значение поля указанным значением. Чтобы указать поле во встроенном документе или в массиве, используйте оператор «.» (точка).

Изменим ФИО второго учащегося группы «Группа по футболу 1» на Степовой А.А. (см. Рисунок 34).

```
db.groups.update({"title":"Группа по футболу 1"}, {$set:
{"pupils.1":"Степовой А.А"}})
```

```
> db.groups.update({"title":"Группа по футболу 1"}, {$set: {"pupils.1":"Степовой А.А"}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Рисунок 34. Изменение данных при помощи update() с оператором $set.

Проверим: (см. Рисунок 35)

```
> db.groups.find({"title": "Группа по футболу 1"}).pretty()
{
        "_id" : ObjectId("5ea6c44b97c79a4351645dab"),
        "title" : "Группа по футболу 1",
        "col" : 20,
        "type" : "Подростковая",
        "pupils" : [
                "Сенин С.С.",
                "Степовой А.А",
                "Андреев К.К."
        ]
}
```

Рисунок 35. Данные после изменения при помощи update() с оператором $set.

2.8.4. Изменение нескольких документов коллекции

Метод коллекции **updateMany()** [25.] обновляет все документы, которые соответствуют указанному фильтру для коллекции.

Метод **updateMany()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.updateMany(<запрос>, <данные, на которые нужно
заменить>, <настройки>)
```

Параметры метода **updateMany()** расшифровываются в Таблица 1.

Таблица 13. Параметры команды updateMany()

Параметр	Тип	Описание
Запрос	Документ	Критерий (запрос), который используется для поиска требуемого документа.
Данные, на которые нужно заменить	Документ	Данные, используемые для замены.
Настройки		
upsert	boolean	Необязательный. Если установлено

Параметр	Тип	Описание
		значение true, создает новый документ, когда ни один документ не соответствует критериям запроса. Значением по умолчанию является false, которое не вставляет новый документ, если совпадение не найдено.
collation	Документ	Необязательный. Сопоставление позволяет пользователям задавать языковые правила для сравнения строк, например правила для букв и знаков ударения.
arrayFilters	Массив	Необязательный. Массив документов, содержащий условия фильтрации, которые определяют, какие элементы массива нужно изменить для операции обновления в поле массива.
hint	Документ или строка	Необязательный. Документ или строка, в которой указывается индекс, используемый для поддержки предиката запроса. Опция может принимать документ спецификации индекса или строку имени индекса.

С методом **updateMany()** используются те же операторы, что и с методом **update()**.

Пример:

Всем тренерам первой категории добавим награду (поле массив awards) (см. Рисунок 36).

```
db.coaches.updateMany({"category": "Первая категория"}, {$addToSet: {"awards":"За особые заслуги в развитии физической культуры и спорта"}})
```

```
> db.coaches.updateMany({"category": "Первая категория"}, {$addToSet: {"awards":"За особые заслуги в развитии физической
 культуры и спорта"}})
{ "acknowledged" : true, "matchedCount" : 3, "modifiedCount" : 3 }
```

Рисунок 36. Изменение данных при помощи updateMany() с оператором $addToSet.

Проверим, изменились ли данные: (см. Рисунок 37)



Рисунок 37. Данные после изменения при помощи updateMany() с оператором $addToSet.

2.8.5. Удаление документов коллекции

Операция **remove** [26.] используется для удаления документов из коллекции.

Метод **remove()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.remove(<запрос>, <только один документ>)
```

Параметры метода **remove()** расшифровываются в Таблица 14.

Таблица 14. Параметры команды remove()

Параметр	Тип	Описание
Запрос	Документ	Обязательный. Критерий (запрос), который используется для поиска требуемого документа.
Только один документ	Boolean	Необязательный. Принимает значение true, если необходимо удалить только один документ коллекции, иначе – false.

Вызов метода «remove» без передачи ему параметров удалит все документы из коллекции.

Пример:

Удалим группу с названием «Группа по плаванию 2». (см. Рисунок 38)

```
db.groups.remove({"title":"Группа по плаванию 2"})
```

```
> db.groups.remove({"title":"Группа по плаванию 2"})
WriteResult({ "nRemoved" : 1 })
```

Рисунок 38. Удаление данных при помощи remove().

2.9. Дополнительные операторы для чтения документов

2.9.1. Сортировка

Метод **sort()** [27.] указывает порядок, в котором запрос возвращает соответствующие документы. Нужно применять метод **sort()** к курсору (возвращает метод find()) перед извлечением любых документов из базы данных. Укажите в параметре сортировки поле или поля для сортировки и значение 1 или -1, чтобы указать сортировку по возрастанию или убыванию соответственно.

Метод **sort()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.find(<запрос>).sort(<параметры сортировки>)
```

Параметры метода **sort()** расшифровываются в Таблица 15.

Таблица 15. Параметры команды sort()

Параметр	Тип	Описание
Параметры сортировки	Документ	Документ, содержащий поля, по которым будет производиться сортировка на результирующем наборе данных.

Пример:

Получим список групп и отсортируем группы в порядке убывания количества мест в них. (см. Рисунок 39)

```
db.groups.find().sort({"col": -1}).pretty()
```



Рисунок 39. Результат сортировки данных при помощи sort().

2.9.2. Работа с операторами count, distinct, limit

2.9.2.1. Подсчет количества документов

Метод **count()** [28.] подсчитывает количество документов в результирующей выборке.

Метод **count()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.count(<запрос>, <настройки>)
```

Параметры метода **count()** расшифровываются в Таблица 16.

Таблица 16. Параметры команды count()

Параметр	Тип	Описание
Запрос	Документ	Необязательный. Критерии выбора запроса.
Настройки	Документ	Необязательный. Дополнительные опции для изменения количества документов.

Документ с опциями содержит следующие поля: (см. Таблица 17)

Таблица 17. Свойства объекта с опциями

Поле	Тип	Описание
limit	Integer	Необязательный. Максимальное количество документов для подсчета.
skip	Integer	Необязательный. Количество документов, пропускаемых перед подсчетом.
hint	string or document	Необязательный. Подсказка или спецификация имени индекса для запроса.

Пример:

Выводим количество тренеров (см. Рисунок 40).

```
db.coaches.count()
```

```
> db.coaches.count()
5
```

Рисунок 40. Результат работы метода count().

Выводим количество тренеров по футболу (см. Рисунок 41).

```
db.coaches.count({"sport": "футбол"})
```

```
> db.coaches.count({"sport": "футбол"})
2
```

Рисунок 41. Результат работы метода count() с фильтрацией.

Ограничим подсчет двумя документами (см. Рисунок 42).

```
db.coaches.count({}, {limit: 2})
```

```
> db.coaches.count({}, {limit: 2})
2
```

Рисунок 42. Результат работы метода count() с ограничением числа записей.

Пропуск первых трех результатов выглядит следующим образом: (см. Рисунок 43)

```
db.coaches.count({}, {skip: 3})
```

```
> db.coaches.count({}, {skip: 3})
2
```

Рисунок 43. Результат работы метода count() с пропуском записей.

2.9.2.2. Вывод уникальных значений

Метод коллекции **distinct()** [29.] находит уникальные значения для указанного поля в одной коллекции или представлении и возвращает результаты в массиве.

Метод **distinct()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.distinct(<поле>, <запрос>)
```

Параметры метода **distinct()** расшифровываются в Таблица 18.

Таблица 18. Параметры команды distinct()

Параметр	Тип	Описание
Поле	Строка	Обязательный. Поле, для которого возвращаются различные значения.
Запрос	Документ	Необязательный. Запрос, который определяет документы в выборке.

Пример:

Выводим уникальные значения видов спорта у тренерского состава (см. Рисунок 44).

```
db.coaches.distinct("sport")
```

```
> db.coaches.distinct("sport")
[ "волейбол", "футбол", "плавание", "легкая атлетика" ]
```

Рисунок 44. Результат работы метода distinct().

Выведем уникальные значения видов спорта у тренерского состава первой категории. (см. Рисунок 45)

```
db.coaches.distinct("sport", {category: "Первая категория"})
```

```
> db.coaches.distinct("sport", {category: "Первая категория"})
[ "волейбол", "плавание", "футбол" ]
```

Рисунок 45. Результат работы метода distinct() с фильтрацией.

2.9.2.3. Ограничение количества документов

Используйте метод **limit()** [30.], чтобы указать максимальное количество документов, которые будет возвращать метод find(). **limit ()** аналогичен инструкции LIMIT в базе данных SQL.

Метод **limit()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.find(<запрос>).limit(<число документов>)
```

Параметры метода **limit()** расшифровываются в Таблица 19.

Таблица 19. Параметры команды limit()

Параметр	Тип	Описание
Число документов	Integer	Обязательный. Количество документов.
Запрос	Документ	Необязательный. Запрос, который определяет документы в выборке.

Пример:

Выведем 2 документа из коллекции тренеров (coaches). (см. Рисунок 46)

```
db.coaches.find().limit(2).pretty()
```

```
> db.coaches.find().limit(2).pretty()
{
        "_id" : ObjectId("5ea6c42997c79a4351645da4"),
        "fio" : "Иванов И.И.",
        "birth_date" : "14.02.1990",
        "category" : "Первая категория",
        "contacts" : {
                "phone" : "7977654123",
                "email" : "ivanov@gmail.com"
        },
        "awards" : [
                "Лучший тренер 2019"
        ],
        "sport" : "волейбол"
}
{
        "_id" : ObjectId("5ea6c42997c79a4351645da5"),
        "fio" : "Петров П.П.",
        "birth_date" : "18.09.1989",
        "category" : "Высшая категория",
        "contacts" : {
                "phone" : "7977614520",
                "email" : "petrov@gmail.com"
        },
        "awards" : [
                "Лучший тренер 2019",
                "Лучший футболист года"
        ],
        "sport" : "Футбол"
}
```

Рисунок 46. Результат работы метода limit().

2.10. Добавление в коллекцию связанных объектов

2.10.1. Создание ссылки на документ

Используя функциональность DBRef, мы можем установить автоматическое связывание между документами. [31.]

Формальный синтаксис DBRef следующий:

```
{ "$ref" : название_коллекции, "$id": значение [, "$db" : название_бд ]}
```

Параметры, используемые при создании DBRef расшифровываются в Таблица 20.

Таблица 20. Параметры, используемые при создании DBRef

Параметр	Тип	Описание
$ref	Строка	Указывает на коллекцию, где хранится связанный документ.
$id	ObjectId	Указывает на значение, которое и будет представлять что-то наподобие внешнего ключа.
$db	Строка	Необязательный. Указывает на базу данных.

2.10.2. Тип данных ObjectId

Тип данных ObjectId используется для хранения ID документа. Запись: ObjectId(<hexadecimal>), где <hexadecimal> – шестнадцатеричное строковое значение

для нового ObjectId. Класс ObjectId является первичным ключом по умолчанию для документа MongoDB и обычно находится в поле _id во вставленном документе. Связь между документами продемонстрирована на Рисунок 47.

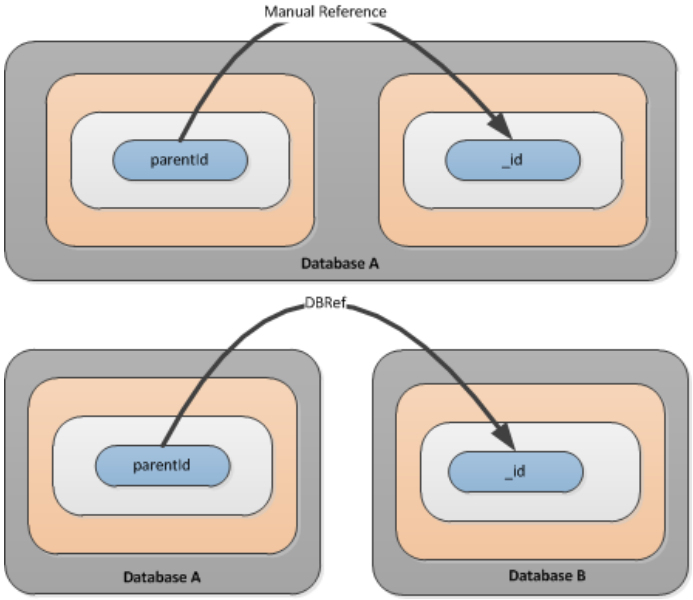


Рисунок 47. Связь между документами при помощи DBRef.

Пример:

Создадим в БД нового тренера. (см. Рисунок 48)

```
      coach = ({"fio": "Степанов В.В.", "birth_date": "20.02.1989", "title"
: "гроссмейстер", "contacts" : {
              "phone" : "8978614544",
              "email" : "stepanov@gmail.com"
          },
          "awards" : [
              "Шахматный Оскар",
              "Лучший шахматист года"
          ],
          "sport" : "шахматы"})
```

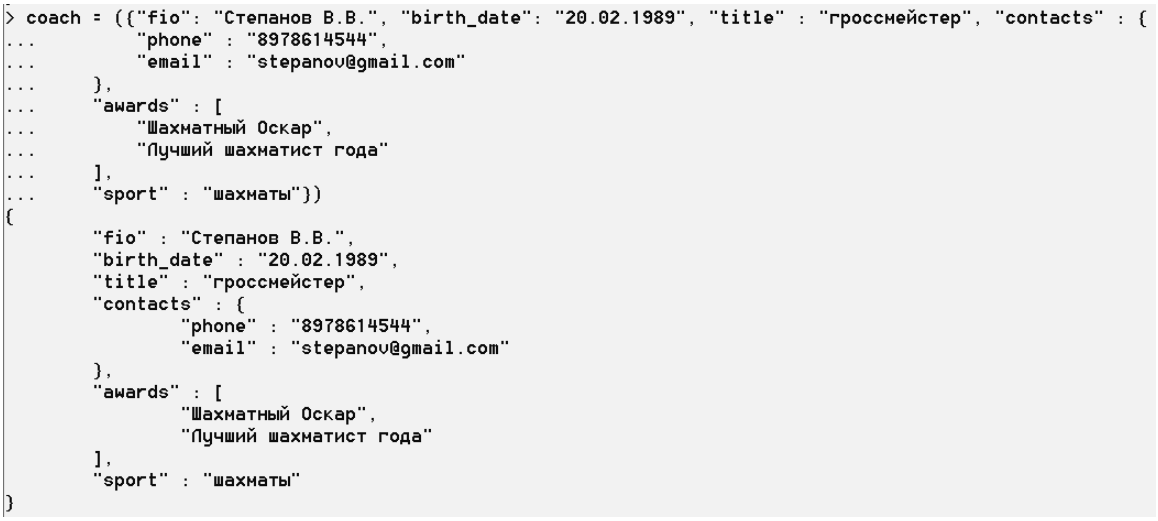


Рисунок 48. Создание объекта нового тренера.

Сохраним объект нового тренера. (см. Рисунок 49)

```
db.coaches.save(coach)
```

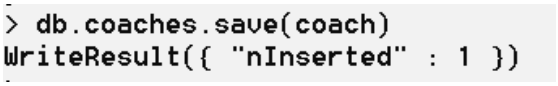


Рисунок 49. Сохранение объекта нового тренера.

Создадим в БД новую группу и свяжем ее с тренером. (см. Рисунок 50)

```
group = ({"title": "Группа по шахматам 1", "col": 10, "type": "Подростковая", "pupils": ["Ивушкин А.Г.", "Иванов И.Р."], "coach": new DBRef('coaches', coach._id)})
```



Рисунок 50. Создание объекта новой группы.

Сохраним объект новой группы. (см. Рисунок 51)

```
db.groups.save(group)
```

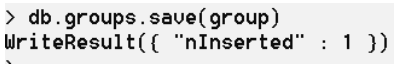


Рисунок 51. Сохранение объекта новой группы.

В данном случае сохранение идет с помощью метода save, а не insert. Метод save при добавлении нового документа генерирует _id.

Выполним переход по связи в запросе.

Находим в коллекции coaches тренера, который работает с группой «Группа по шахматам 1». (см. Рисунок 52)

```
db.coaches.find({_id: db.groups.findOne({"title": "Группа по шахматам 1"}).coach.$id}).pretty()
```

```
> db.coaches.find({_id: db.groups.findOne({"title": "Группа по шахматам 1"}).coach.$id}).pretty()
{
        "_id" : ObjectId("5ea7368fb775bab737344d57"),
        "fio" : "Степанов В.В.",
        "birth_date" : "20.02.1989",
        "title" : "гроссмейстер",
        "contacts" : {
                "phone" : "8978614544",
                "email" : "stepanov@gmail.com"
        },
        "awards" : [
                "Шахматный Оскар",
                "Лучший шахматист года"
        ],
        "sport" : "шахматы"
}
```

Рисунок 52. Переход по связи от группы к тренеру.

2.11. Группировка и агрегирование в MondoDB

Для агрегации в mongo используется отдельная функция - aggregate.

В классическом SQL агрегация производится с помощью группировки и агрегаторов:

```
SELECT column_name, aggregate_function(column_name) FROM table_name
WHERE column_name operator value GROUP BY column_name;
```

В данном примере кортежи будут сгруппированы по column_name и над каждой группой будет применена aggregate_function.

Можно выделить несколько этапов запроса:

- Выборка нужных записей
- Группировка выбранных записей
- Применение агрегационной функции к группам

В MongoDB используется подобный механизм. В начале необходимо указать условие выборки, затем условие группировки, возможно некоторые дополнительные этапы, например сортировки и проекции. Все эти этапы собираются в массив, который называется pipeline. Документы коллекции проходят через pipeline и на выходе получается результат агрегации. [32.]

2.11.1. Комбинация этапов при группировке

Как было сказано выше, этапы агрегации складываются в pipeline, который затем и проходят документы коллекции.

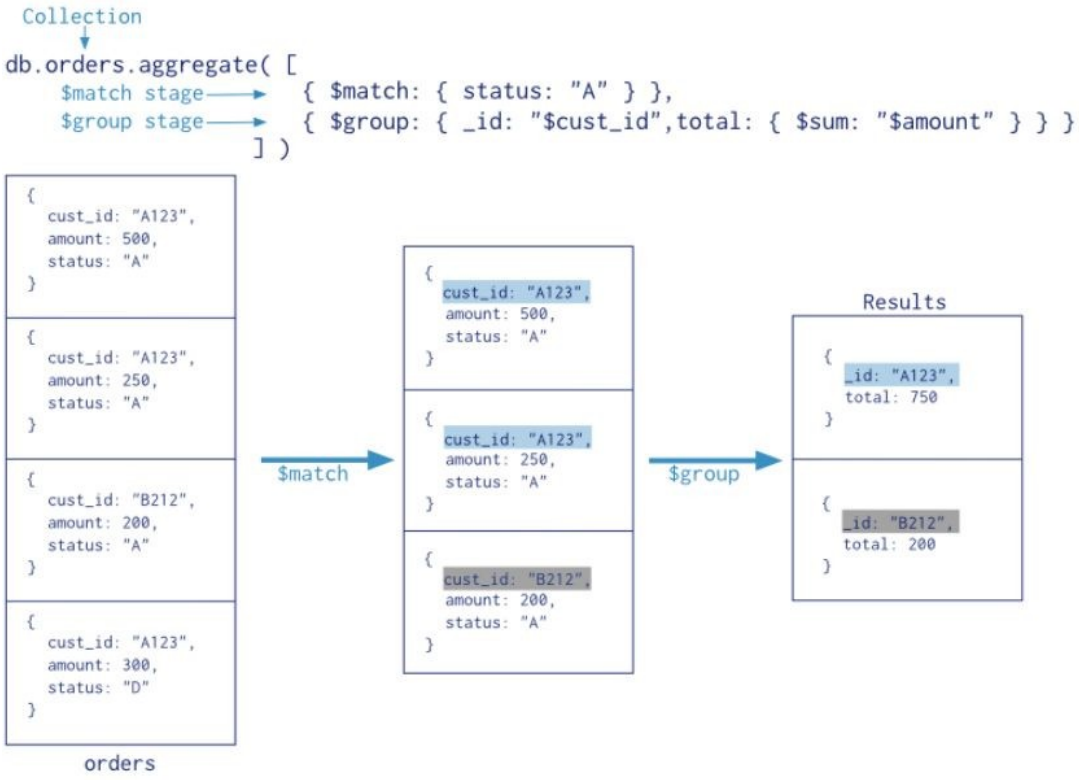


Рисунок 53. Пример выполнения комбинированной агрегации.

На рисунке (см. Рисунок 53) рассматривается пример агрегации, где сначала выбираются документы, в которых поле статус = “A”, а затем группируются по cust_id и для каждой группы суммируются значения из поля amount.

2.11.2. Агрегирование данных

Метод **aggregate()** [33.] имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.aggregate(<конвейер>, <настройки>)
```

Параметры метода **aggregate()** расшифровываются в Таблица 21.

Таблица 21. Параметры команды aggregate()

Параметр	Тип	Описание
Конвейер	Массив	Обязательный. Последовательность операций или этапов агрегирования данных.
Настройки	Документ	Необязательный. Дополнительные параметры.

2.11.3. Этапы агрегирования данных

Этапы агрегирования данных [34.] 34.представлены в Таблица 22.

Таблица 22. Этапы агрегирования данных

Оператор	Общий синтаксис		Параметры	Описание
$project	<field>: <1 or true>	Определяет включение поля.	Integer, Boolean, Документ	Используется для выбора некоторых специальных полей из коллекции.
	_id: <0 or false>	Определяет включение поля _id.		
	<field>: <expression>	Добавляет новое поле или сбрасывает значение существующего поля.		
	<field>:<0 or false>	Определяет исключение поля.		
$count	{ $count: <string> }		Строка	Возвращает счетчик количества документов на этом этапе конвейера агрегации.
$sort	{ $sort: { <field1>: <sort order>, <field2>: <sort order> ... } }		Документ	Фильтрует поток документов, чтобы только соответствующие документы могли пройти без изменений на следующем этапе конвейера.
$match	{ $match: { <query> } }		Документ	$avg, $push и т.д. Непосредственно, сама агрегация.

Оператор	Общий синтаксис	Параметры	Описание
$group	{ $group: { _id: <expression>, <field1>: { <accumulator1> : <expression1> }, ... } }	Документ	В поле _id нужно передать выражение, по которому будет происходить группировка, например, это может быть название поля. Дальше в документе перечисляются поля, которые будут содержать агрегированные значения, которые получаются с помощью аккумуляторов, таких как $sum, $max, $min,
$limit	{ $limit: <positive integer> }	Integer	Сортирует все входные документы и возвращает их в конвейер в отсортированном порядке.
$skip	{ $skip: <positive integer> }	Integer	Ограничивает количество документов, передаваемых на следующий этап в конвейере.
$unwind	{ $unwind: <field path> }	$поле объекта	С помощью данной команды можем проигнорировать список документов в имеющемся множестве.

2.11.4. Список операций для агрегации документов

Операции для агрегации документов, использующиеся с **$group** [35.], представлены в Таблица 23.

Таблица 23. Операции для агрегации документов, использующиеся с $group

Оператор	Общий синтаксис	Параметры		Описание
$max	{$max: <expression> } {$max: [<expression1>, <expression2> ...] }	$название_поля [$название_поля_1, $название_поля_2, …]		Возвращает самое большое значение выражения для каждой группы.
$min	{$min: <expression> } {$min: [<expression1>, <expression2> ...] }	$название_поля [$название_поля_1, $название_поля_2, …]		Возвращает самое маленькое значение выражения для каждой группы.
$sum	{$sum: <expression> } {$sum: [<expression1>, <expression2> ...] }	Число	Вернет сумму чисел	Возвращает сумму числовых значений. Игнорирует нечисловые значения.
		Число и не число	Вернет сумму числовых значений	
		Не число и не существующее значение	Вернет 0	
$avg	{$avg: <expression> } {$avg: [<expression1>, <expression2> ...] }	$название_поля [$название_поля_1, $название_поля_2, …]		Возвращает среднее числовых значений. Игнорирует нечисловые значения.
$first	{$first: <expression> }	$название_поля		Взять первый элемент группы.
$last	{$last: <expression> }	$название_поля		Взять последний элемент группы.

2.11.5. Примеры агрегации

Запрос вернет документы из коллекции groups (группы), сгруппированные по типу. (см. Рисунок 54)

```
db.groups.aggregate([
    {
        $group: {
            _id: "$type"
        }
    }
])
```



Рисунок 54. Группировка спортивных групп по типу.

Этот запрос вернет документы из коллекции groups (группы), сгруппированные по типу и для каждого типа будет посчитано суммарное число мест в группах. (см. Рисунок 55)

```
db.groups.aggregate([
    {
        $group: {
            _id: "$type",
            sum_col: {$sum: "$col"}
        }
    }
])
```

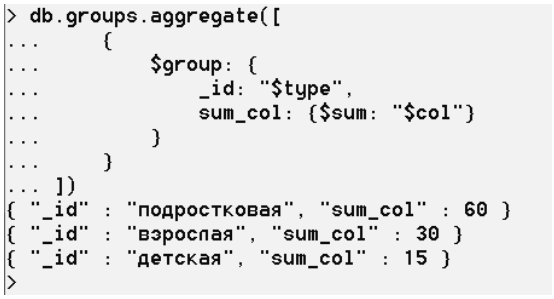


Рисунок 55. Группировка спортивных групп по типу и подсчет суммы мест в группах.

Запрос вернет документы из коллекции groups (группы), сгруппированные по типу и для каждого типа будет посчитано суммарное число мест в группах, которое будет отсортировано по убыванию. (см. Рисунок 56)

```
db.groups.aggregate([
    { $group: { _id: "$type", sum_col: { $sum: "$col" } } },
    { $sort: { sum_col: -1 } }
])
```

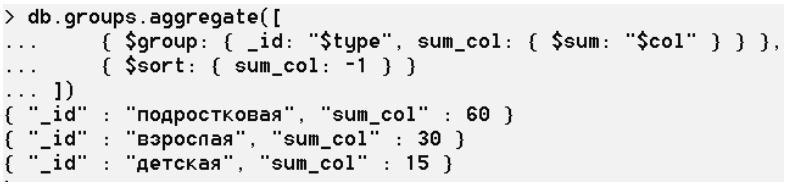


Рисунок 56. Группировка спортивных групп по типу, подсчет суммы мест в группах и сортировка посчитанных значений по убыванию.

2.12. Работа с индексами в MongoDB

2.12.1. Создание и использование индекса

Индексы [36.] позволяют упорядочить данные по определенному полю, для которого введен индекс, что впоследствии ускорит поиск. С помощью метода **createIndex** [37.] устанавливается индекс по заданному полю. MongoDB позволяет установить до 64 индексов на одну коллекцию.

Метод **createIndex()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.createIndex(<ключи>, <настройки>)
```

Параметры метода **createIndex()** расшифровываются в Таблица 24.

Таблица 24. Параметры команды createIndex()

Параметр	Тип	Описание
Ключи	Документ	Обязательный. Документ, который содержит пары полей и значений, где поле является ключом индекса, а значение описывает тип индекса для этого поля. Для восходящего индекса в поле укажите значение 1; для нисходящего индекса укажите значение -1.
Настройки	Документ	Необязательный. Документ, содержащий набор параметров, управляющих созданием индекса.

Документ с настройками содержит поля (см. Таблица 25):

Таблица 25. Поля документа с настройками для команды createIndex()

Параметр	Тип	Описание
background	boolean	Значение true указывает MongoDB на создание индекса в фоновом режиме. Фоновые сборки не блокируют операции над коллекцией. По умолчанию false.
unique	boolean	Необязательный. Создает уникальный индекс, чтобы коллекция не принимала вставку или обновление документов, если значение ключа индекса совпадает с существующим значением в индексе. По умолчанию false.
name	string	Необязательный. Название индекса. Если не указано, MongoDB генерирует имя индекса, объединяя имена проиндексированных полей и порядок сортировки.
partialFilterExpression	document	Необязательный. Если указано, индекс ссылается только на документы, которые соответствуют выражению фильтра($exists, $gt, $lt и др.).
expireAfterSeconds	integer	Необязательный. Задает значение в секундах в виде TTL для управления продолжительностью хранения документов в этой коллекции MongoDB.

Параметр	Тип	Описание
storageEngine	document	Необязательный. Позволяет пользователям настраивать механизм хранения для каждого индекса при создании индекса.

Пример:

Введем индекс для поля title (название) в коллекции groups (группы). (см. Рисунок 57)

```
db.groups.createIndex({"title": 1})
```

```
> db.groups.createIndex({"title": 1})
{
        "createdCollectionAutomatically" : false,
        "numIndexesBefore" : 1,
        "numIndexesAfter" : 2,
        "ok" : 1
}
```

Рисунок 57. Создание индекса.

2.12.2. Настройка индексов

Если мы просто определим индекс для коллекции, например, db.groups.createIndex({"title": 1}), то мы все еще сможем добавлять в коллекцию документы с одинаковым значением ключа title. Однако, если нам потребуется, чтобы в коллекцию можно было добавлять документ с одним и тем же значением ключа только один раз, мы можем установить флаг unique.

Пример:

Введен уникальный индекс для поля название (title) в коллекции групп. (см. Рисунок 58)

```
db.groups.createIndex({"title": 1}, {"unique" : true})
```

```
> db.groups.createIndex({"title": 1}, {"unique" : true})
{
        "createdCollectionAutomatically" : false,
        "numIndexesBefore" : 1,
        "numIndexesAfter" : 2,
        "ok" : 1
}
```

Рисунок 58. Создание уникального индекса.

Индекс можно задать для нескольких полей. Однако, значение поля, по которому идет индексация, не должно быть больше 1024 байт.

2.12.3. Управление индексами

Можно воспользоваться методом **getIndexes** и вывести всю информацию об индексах для конкретной коллекции.

Метод **getIndexes ()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.getIndexes()
```

Пример:

Выведем все индексы для коллекции групп. (см. Рисунок 59)

```
db.groups.getIndexes()
```

```
> db.groups.getIndexes()
[
        {
                "v" : 2,
                "key" : {
                        "_id" : 1
                },
                "name" : "_id_",
                "ns" : "sport_school_db.groups"
        },
        {
                "v" : 2,
                "unique" : true,
                "key" : {
                        "title" : 1
                },
                "name" : "title_1",
                "ns" : "sport_school_db.groups"
        }
]
```

Рисунок 59. Вывод всех индексов для коллекции.

Здесь для коллекции groups (из БД sport_school_db) определено 2 индекса: id и title. Поле key используется для поиска максимального и минимального значений, для различных операций, где надо применять данный индекс. Поле title применяется в качестве идентификатора для операций администрирования.

Выведем информацию о выполнении для указанного запроса: поиск по полю, для которого введен индекс. При поиске по индексу просматривается один документ.

Метод **explain()** [38.] возвращает документ с планом запроса и, необязательно, статистикой выполнения. ExecutionStats включает в себя информацию о выполнении завершенного запроса для оптимального плана выполнения запроса.

Метод **explain()** имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.explain(<verbose>)
```

Параметры метода **explain()** расшифровываются в Таблица 26.

Таблица 26. Параметры команды explain()

Параметр	Тип	Описание
verbose	Строка	Необязательный. Определяет режим для вывода информации о выполнении запроса. Режим влияет на поведение explain() и определяет объем возвращаемой информации. Возможные режимы: "queryPlanner", "executionStats" и "allPlansExecution". По умолчанию установлен "queryPlanner".

2.12.4. Удаление индекса

Метод **dropIndex()** [39.] имеет следующий синтаксис:

```
db.<ИМЯ_КОЛЛЕКЦИИ>.dropIndex(<индекс>)
```

Параметры метода **dropIndex()** расшифровываются в Таблица 27.

Таблица 27. Параметры команды dropIndex()

Параметр	Тип	Описание
Индекс	Строка или документ	Необязательный. Указывает индекс для удаления. Можно указать индекс либо по имени индекса, либо по документу спецификации индекса.

Пример:

Удалим индекс для поля название в коллекции групп. (см. Рисунок 60)

```
db.groups.dropIndex("title_1")
```

```
> db.groups.dropIndex("title_1")
{ "nIndexesWas" : 2, "ok" : 1 }
```

Рисунок 60. Удаление индекса.

Выведем информацию о выполнении для указанного запроса: поиск по полю, для которого не введен индекс. При поиске по полю, для которого не введен индекс просматриваются все документы коллекции groups (группы) – 5 штук. (см. Рисунок 61)

```
db.groups.find({"title": "Группа по шахматам
1"}).explain("executionStats").executionStats
```

```
"executionSuccess" : true,
"nReturned" : 1,
"executionTimeMillis" : 0,
"totalKeysExamined" : 0,
"totalDocsExamined" : 5,
```

Рисунок 61. Статистика выполнения запроса.

2.13. Режимы для вывода информации о выполнении запроса

Режимы для вывода информации о выполнении запроса представлены в Таблица 28.

Таблица 28. Режимы для вывода информации о выполнении запроса

Название режима	Описание
queryPlanner	Установлен по умолчанию. MongoDB запускает оптимизатор запросов, чтобы выбрать оптимальный план для оцениваемой операции.
executionStats	MongoDB запускает оптимизатор запросов, чтобы выбрать оптимальный план, выполняет выбранный оптимальный план до его завершения и возвращает статистику, описывающую выполнение выбранного плана.
allPlansExecution	В режиме «allPlansExecution» MongoDB возвращает статистику, описывающую выполнение оптимального плана выполнения запроса, а также статистику для других планов-кандидатов, определенных во время выбора плана.

Пример:

Получим статистику выполнения запроса поиска по полю, для которого введен индекс. (см. Рисунок 62)

```
db.groups.find({"title": "Группа по шахматам 1"}).explain("executionStats").executionStats
```

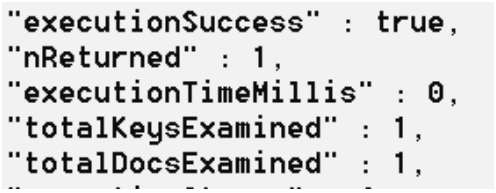


Рисунок 62. Статистика выполнения запроса.

2.14. Дополнительные возможности

2.14.1. Организация и виды кластеров

Кластер [40] — это разновидность параллельной или распределённой системы, которая:

- состоит из нескольких связанных между собой компьютеров;
- используется как единый, унифицированный компьютерный ресурс.

Обычно различают следующие основные виды кластеров:

1. отказоустойчивые кластеры (High-availability clusters, HA, кластеры высокой доступности)
2. кластеры с балансировкой нагрузки (Load balancing clusters)
3. вычислительные кластеры (High performance computing clusters, HPC)
4. системы распределенных вычислений.

2.14.1.1. Кластеры высокой доступности

Создаются для обеспечения высокой доступности сервиса, предоставляемого кластером. Избыточное число узлов, входящих в кластер, гарантирует предоставление сервиса в случае отказа одного или нескольких серверов. Типичное число узлов — два, это минимальное количество, приводящее к повышению доступности. Создано множество программных решений для построения такого рода кластеров.

Отказоустойчивые кластеры и системы разделяются на 3 основных типа:

- с холодным резервом или активный/пассивный. Активный узел выполняет запросы, а пассивный ждет его отказа и включается в работу, когда таковой произойдет. Пример — резервные сетевые соединения, в частности, Алгоритм связующего дерева. Например, связка DRBD и HeartBeat/Corosync.
- с горячим резервом или активный/активный. Все узлы выполняют запросы, в случае отказа одного нагрузка перераспределяется между оставшимися. То есть кластер распределения нагрузки с поддержкой перераспределения запросов при отказе. Примеры — практически все кластерные технологии, например, Microsoft Cluster Server. OpenSource проект OpenMosix.
- с модульной избыточностью. Применяется только в случае, когда простой системы совершенно недопустим. Все узлы одновременно выполняют один и тот же запрос (либо части его, но так, что результат достижим и при отказе любого узла), из результатов берется любой. Необходимо гарантировать, что результаты разных узлов всегда будут одинаковы (либо различия гарантированно не повлияют на дальнейшую работу). Примеры — RAID и Triple modular redundancy.

Конкретная технология может сочетать данные принципы в любой комбинации. Например, Linux-HA поддерживает режим обоюдной поглощающей конфигурации (англ. takeover), в котором критические запросы выполняются всеми узлами вместе, прочие же равномерно распределяются между ними.

2.14.1.2. Кластеры с балансировкой нагрузки

Принцип их действия строится на распределении запросов через один или несколько входных узлов, которые перенаправляют их на обработку в остальные, вычислительные узлы. Первоначальная цель такого кластера — производительность, однако, в них часто используются также и методы, повышающие надёжность. Подобные конструкции называются серверными фермами. Программное обеспечение (ПО) может быть как коммерческим (OpenVMS, MOSIX, Platform LSF HPC, Solaris Cluster, Moab Cluster Suite, Maui Cluster Scheduler), так и бесплатным (OpenMosix, Sun Grid Engine, Linux Virtual Server).

2.14.1.3. Вычислительные кластеры

Кластеры используются в вычислительных целях, в частности в научных исследованиях. Для вычислительных кластеров существенными показателями являются высокая производительность процессора в операциях над числами с плавающей точкой (flops) и низкая латентность объединяющей сети, и менее существенными — скорость операций ввода-вывода, которая в большей степени важна для баз данных и web-сервисов. Вычислительные кластеры позволяют уменьшить время расчетов, по сравнению с одиночным компьютером, разбивая задание на параллельно выполняющиеся ветки, которые обмениваются данными по связывающей сети. Одна из типичных конфигураций — набор компьютеров, собранных из общедоступных компонентов, с установленной на них операционной системой Linux, и связанных сетью Ethernet, Myrinet, InfiniBand или другими относительно недорогими сетями. Такую систему принято называть кластером Beowulf. Специально выделяют высокопроизводительные кластеры (Обозначаются англ. аббревиатурой HPC Cluster — High-performance computing cluster).

2.14.1.4. Системы распределенных вычислений

Такие системы не принято считать кластерами, но их принципы в значительной степени сходны с кластерной технологией. Их также называют grid-системами. Главное отличие — низкая доступность каждого узла, то есть невозможность гарантировать его работу в заданный момент времени (узлы подключаются и отключаются в процессе работы), поэтому задача должна быть разбита на ряд независимых друг от друга процессов. Такая система, в отличие от кластеров, не похожа на единый компьютер, а служит упрощённым средством распределения вычислений. Нестабильность конфигурации, в таком случае, компенсируется большим числом узлов.

2.14.2. Репликация

Репликация [41.] – это процесс синхронизации данных на нескольких серверах. Данный механизм уменьшает затраты ресурсов и увеличивает доступность данных, копии которых хранятся на разных серверах. Репликация защищает базу данных (далее – БД) от потери единственного сервера и позволяет сохранить данные в случае технической неисправности на одном из серверов. [42.]

Рассмотрим стандартную схему асинхронной репликации на примере MySQL (см. Рисунок 63).

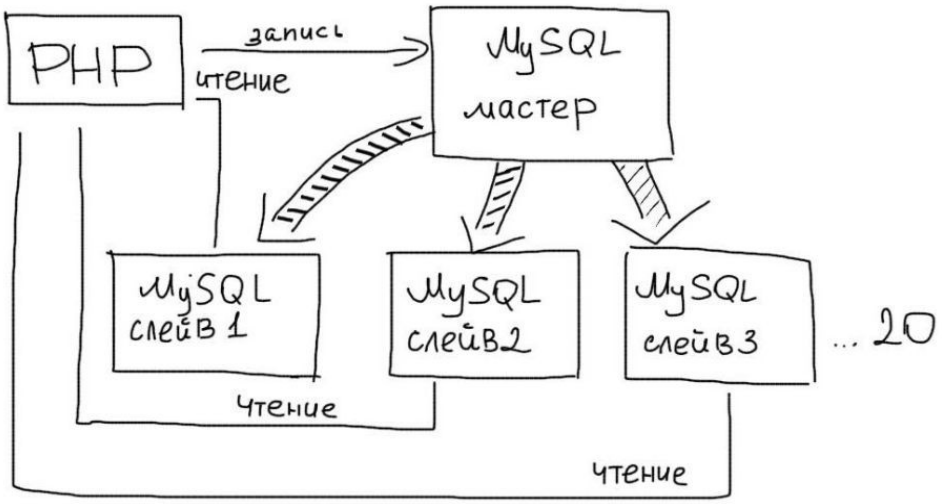


Рисунок 63. Схема репликации MySQL.

В этой схеме есть 2 типа узлов:

- Мастер-узел - используется для записи
- Слейв-узел - используется для чтения, дублирует записи мастер-узла.

В MongoDB используется похожая схема.

В MongoDB репликация асинхронная. Это значит, что данные синхронизируются между репликами не в момент непосредственного изменения данных, а отложенно, через какое-то время. В этом есть плюс: не тратится время на репликацию в момент изменения данных (insert'ы происходят быстрее). В определенные моменты времени данные между репликами могут быть не согласованными.

В MongoDB существуют replicaset сервера. К реплицированным кластерам мы можем подключаться и даже распределять нагрузку по чтению. Под репликацией понимается размещение и обслуживание серверов базы данных на нескольких машинах. В MongoDB реализовано две модели репликации:

- главный-подчиненный (Master-Slave)
- наборы реплик (Replica Set).

В обоих случаях единственный первичный узел выполняет все операции записи, после чего вторичные узлы считывают описания этих операций и асинхронно применяют их у себя. В основном используют паттерн наборы реплик, потому что он позволяет дополнительно к основным возможностям, еще и восстановить или развернуть новую базу из реплики. Единственный случай когда мы можем применять главный-подчиненный это случай если у нас больше 11 подчиненных, потому что в наборе реплик может быть только 12 членов.

Основное назначение репликации — обеспечить резервирование. Это означает, что должна поддерживаться связь между узлом и его репликами. Важно отметить, что хотя реплики и обеспечивают резервирование, они далеки от резервного копирование. Резервное копирование — это снимок базы на определенный момент времени, а реплика всегда актуальна. Модели репликации представлены на Рисунок 64.

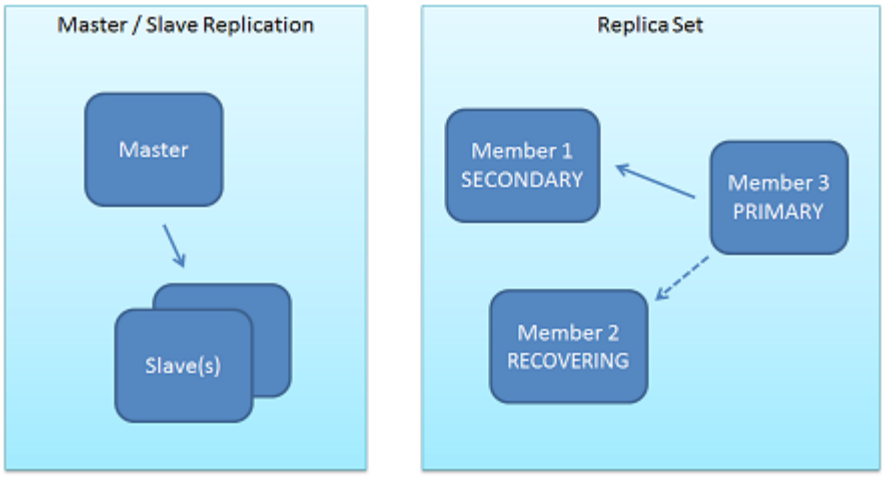


Рисунок 64. Модели репликации: Master-Slave (слева) и Replica Set (справа).

2.14.2.1. Как работает репликация

Набор реплик основан на двух механизмах: Журнал операций (oplog) и тактовый сигнал (heartbeat). Журнал операций делает репликацию возможной, а с помощью тактовых сигналов ведется мониторинг состояния и активируется процедура обработки отказа.

2.14.2.2. Журнал операций

Это сердце репликаций. Он представляет собой ограниченную коллекцию в базе local на каждом узле, в эту коллекцию записываются все изменения данных, достаточное количество для воспроизведения операции.

2.14.2.3. Тактовые сигналы

Механизм тактовых сигналов позволяет обеспечить отработку отказа и выбор нового первичного узла. По умолчанию каждый член набор реплик посылает тактовые сигналы всем остальным членам раз в две секунды. Это позволяет в целом отслеживать состояние системы. Если узел перестает отвечать и является первичным, то система автоматически выбирает новый первичный узел из вторичных с самым свежим состоянием.

2.14.2.4. Фиксация и откат

Мы можем писать в первичный узел весь день напролет, но данные не будут считаться зафиксированными пока они не будут реплицированы в большинство реплик. Представим такую ситуацию. Мы записали данные в первичный узел, но они не успели реплицироваться на вторичный, из-за сбоев в сети, например, и неожиданно главный сервер выходит из строя, вторичный сервер становиться первичным и в него производятся операции записи, но рано или поздно бывший первичный сервер восстановиться и попытается реплицировать данные с нового первичного узла. Проблема в том, что на старом первичном узле имеются записи, которых нет в новом первичном. В такой ситуации инициируется откат. При откате все операции записи, которые не были реплицированы на большинство узлов, отменяются, то есть удаляются из журнала операций вторичного узла из коллекции, которой адресованы. Если мы удалили документ и его следует откатить, то база найдет данный документ в другой реплике и восстановит его.

2.14.2.5. Модель репликации Master-Slave

Конфигурация репликации Master-Slave [42.43.] проста в настройке. Перед настройкой репликации Master-Slave в MongoDB необходимо точно знать, что такое master, slave, набор реплик и их различие. База данных, которая получает все записи в обычной репликации ведущий-ведомый, называется главной. База данных только для чтения, которая реплицирует операции из базы данных master в обычной репликации master / slave, называется slave. Набор реплик - это группа процессов mongod, которые

поддерживают один и тот же набор данных. Основное различие между репликацией «ведущий-ведомый» и набором реплик состоит в том, что набор реплик имеет параметры автоматического переключения при сбое. Репликация Master-Slave имеет опции аварийного переключения вручную.

Рассмотрим пример, где есть несколько серверов: один мастер, и два слейва. Удалить и писать в базу можно только в мастер-сервере, а читать как из мастера так и из слейвов (для этого-то они и нужны). Слейвов может быть несколько. Такая схема полезна, когда у нашей системы много запросов на получение данных. Так как приложение может обращаться к слейвам для чтения, то мы можем более менее равномерно распределить нагрузку между ними.

2.14.2.6. Настройка модели репликации master-slave в MongoDB

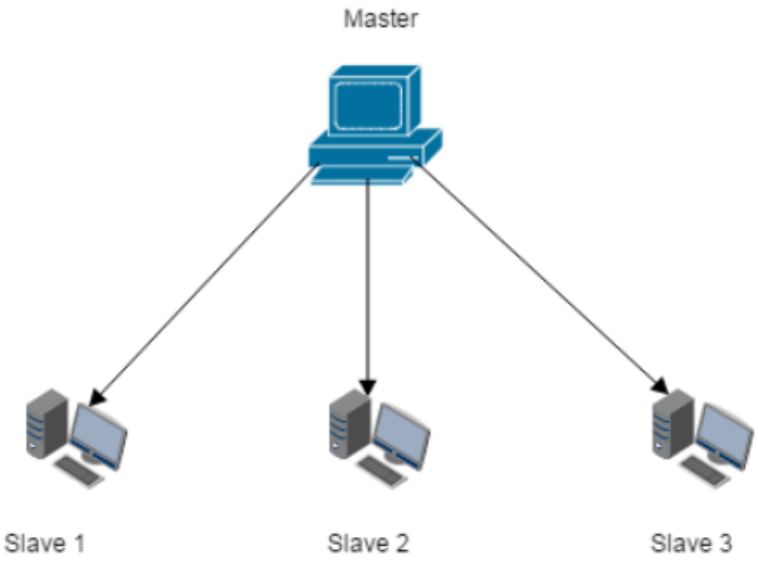


Рисунок 65. Модель репликации master-slave в MongoDB с одним мастером и тремя подчиненными.

Теперь посмотрим, как реализовать master-slave в MongoDB. Допустим у нас три машины: 127.0.0.1 будет мастером, а 127.0.0.2 и 127.0.0.3 соответственно слейвами. Для начала запустим mongod в режиме мастера:

```
mongod --master --dbpath /data/masterdb/
```

Запустим мастер-процесс:

```
$bin/mongod --master
```

С опцией --master mongod создаст коллекцию local.oplog.$main, которая представляет собой «журнал операций», который ставит в очередь операции, которые подчиненные будут применять для репликации операций от мастера. --dbpath не является обязательным, указывает путь к БД.

Теперь на нашей первой машине запущен мастер, к которому можно обращаться для записи/чтения уже сейчас. Теперь на двух других серверах запустим mongod в режиме слейвов:

```
mongod --slave --source <masterhostname><:<port>> --dbpath
/data/slavedb/
```

Укажите имя хоста и порт главного экземпляра в аргументе --source. --Dbpath не является обязательным параметром.

Для ведомых экземпляров MongoDB хранит данные об исходном сервере в коллекции local.sources.

В нашем случае:

```
$bin/mongod --slave --source 127.0.0.1: 27017
$bin/mongod --slave --source 127.0.0.1:27017
```

Запущенные слейвы знают, где находится мастер (параметр source). Если в мастер-сервер кто-то что-то писал/удалял пока мы настраивали слейвы, то слейвы автоматически и асинхронно синхронизируются с мастером, и у них будет свежая копия данных.

Для того, чтобы понять какие сложности могут возникнуть со слейвом, надо понять как работает мастер. Когда мы запустили мастера-сервер, он создал в своей базе коллекцию local.oplog.$main (лог операций). В этой коллекции хранятся все последние операции (изменения данных), которые применялись на мастере. Именно из этой коллекции по слейвам расходятся команды обновления их локальных копий. Размер лога операций ограничен (мы его можем задать сами). И в случае, если лог будет заполнен полностью, более старые операции будут удаляться из него, а новые соответственно добавляться.

Дело в том, что если слейв теряет ненадолго работоспособность, то есть вероятность, что на мастере за это время будет применено столько новых операций, что

лог заполнится и самые старые операции удалятся. А это значит, что мастер не сможет отослать введенному в строй слейву весь лог изменений и на слейве окажется не синхронная копия мастера. А это очень плохо. В этом случае поможет полное копирование базы мастера на слейв. Это можно сделать операцией {resync:1} на слейве, или запуском слейва с ключом --autoresync. После этого на слейве будет точная копия мастера и наша система снова в строю.

Администрирование master-slave репликации в MongoDB простое, отчасти потому что предоставляется хороший API. Например, можно указывать мастер-сервер на уже работающем слейве или выполнять диагностические команды и др. Достаточно хороша идея автоматического администрирования репликаций: добавление новых слейвов, бэкапы, подмена мастера и прочее.

2.14.2.7. Модель репликации Replica Set

На данный момент MongoDB достигает репликации с помощью набора реплик (ReplicaSet) [44.]. Набор реплик – это группа экземпляров mongod**,** которые содержат один и тот же набор данных. В RelicaSet всегда есть Primary и Secondary узлы. Primary используются для записи аналогично мастер-узлам в стандартной модели репликации, а secondary аналогично slave-узлам. Все данные реплицируются на вторичные узлы. Набор реплик – это группа из двух или более узлов (обычно требуется минимум 3 узла). Модель репликации ReplicaSet приведена на схеме ниже (см. Рисунок 66).

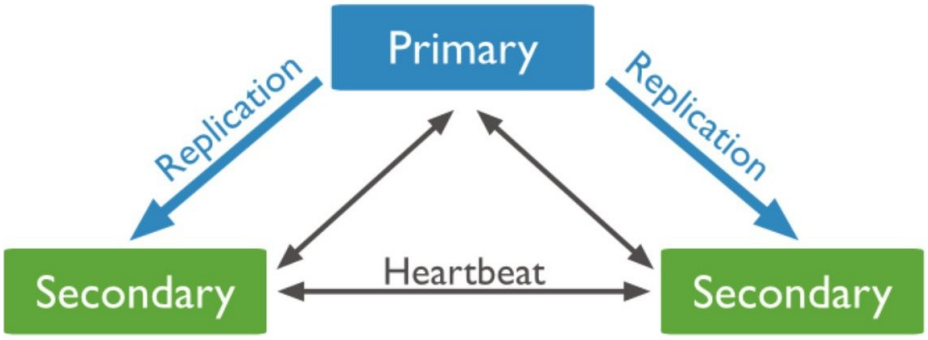


Рисунок 66. Модель репликации ReplicaSet.

Secondary-узлы реплицируют данные Primary-узла, при этом обмениваясь специальными сигналами Heartbeat. Если один из узлов перестал отвечать на hearbeat, значит это отказавший узел. Если отказывает secondary-узел, то никаких проблем не возникает. В случае отказа Primary-узла весь кластер становится недоступным для записи, поэтому необходим новый Primary-узел. Пример отказа Primary-узла приведен на рисунке ниже (см. Рисунок 67).

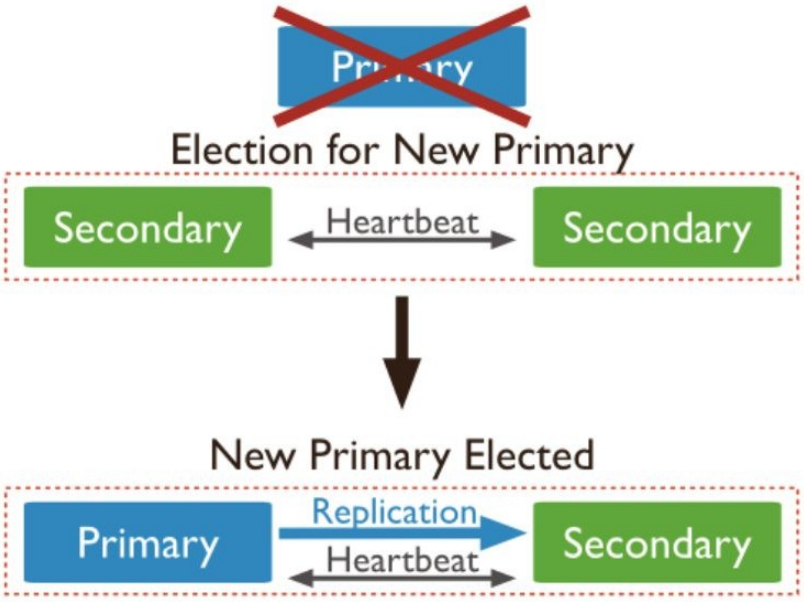


Рисунок 67. Пример отказа Primary-узла.

Определение нового primary-узла в mongo происходит с помощью специального механизма “выборов”. Каждый из оставшихся узлов голосует за один из узлов, чтобы тот стал primary. Узел, набравший кворум голосов, становится Primary-узлом.

Отсюда следует, что минимальным работоспособным кластером будет replicaSet из трех узлов. Получается, что отказоустойчивость кластера = N/2 - 1, например, если кластер состоит из 6 узлов, то для выбора нового Primary-узла необходимо хотя бы 4 работоспособных узлов, чтобы набрать кворум голосов. Значит только 2 узла могут отказать. Поэтому рекомендуется выбирать количество узлов нечетным.

Кроме secondary узлов в голосовании может участвовать еще один типа узла - Arbiter. Он не хранит данных, но может голосовать. Таким образом решается проблема необходимости большого числа голосов, так как не всегда оптимально делать множество реплик.

При конфигурировании replica set’а можно указывать приоритеты становления primary-узлом. Узел с нулевым приоритетом никогда не станет primary.

Кроме этого существует возможность указывать приоритеты для чтения данных. Например, можно указать, что чтение должно производиться только с primary, тогда secondary-узлы фактического работают в режиме бекапа. Узел для чтения также можно выбирать в зависимости от географического положения сервера и клиента. Это может уменьшить накладные расходы передачи данных по сети.

2.14.2.8. Настройка модели репликации ReplicaSet в MongoDB

Минимальная настройка реплик включает в себя 3 узла. 2 экземпляра mongod и третий играет роль арбитра. Начнем с создания каталогов для данных:

```
mkdir /data/node1
mkdir /data/node2
mkdir /data/arbiter
```

Запустим для каждой папки свой процесс mongod:

```
$ mongod --replSet <setname> --dbpath /data/slavedb/ --port <port>
```

--replSet <setname> – replSetName или --replSet для получения информации об установке имени набора реплик.

--dbpath <path> – Каталог, в котором экземпляр mongod хранит свои данные.

--port <port> – Порт TCP, на котором экземпляр MongoDB прослушивает клиентские подключения.

```
mongod --replSet myapp --dbpath /data/node1 --port 40000
mongod --replSet myapp --dbpath /data/node2 --port 40001
mongod --replSet myapp --dbpath /data/arbiter --port 40002
```

Нам нужно сконфигурировать набор реплик, для этого подключимся к одному из запущенных экземпляров, но не к арбитру.

```
mongo --port 40000
```

После подключения выполним команду:

```
rs.initiate(конфигурация)
```

Инициирует набор реплик. При желании метод может принимать аргумент в форме документа, который содержит конфигурацию набора реплик.

Метод **rs.initiate ()** [45.] имеет следующий параметр:

Конфигурация – Необязательный. Документ, который определяет конфигурацию для нового набора реплик. Если конфигурация не указана, MongoDB использует конфигурацию набора реплик по умолчанию.

```
rs.initiate ()
```

Примерно через минуту вы получите набор реплик с одним членом. Чтобы добавить два других, нужно выполнить команду **rs.add()** [46.], если вы ставите локальную репликацию, то могут возникнуть проблемы с ip, поэтому при возникновении проблем используйте другой адрес:

```
rs.add(хост, толькоАрбитр)
```

Добавляет участника в набор реплик. Для запуска метода необходимо подключиться к первичному набору реплик.

Параметры метода **add()** расшифровываются в Таблица 29.

Таблица 29. Параметры команды add()

Параметр	Тип	Описание

хост	Строка или документ	Обязательный. Новый элемент для добавления в набор реплик. Укажите либо в виде строки, либо в виде документа конфигурации. Задает имя хоста и, необязательно, номер порта члена-арбитра, добавляемого в набор реплик.
		`{`
		`_id: <int>,`
		`host: <string>, // required`
		`arbiterOnly: <boolean>,`
		`buildIndexes: <boolean>,`
		`hidden: <boolean>,`
		`priority: <number>,`
		`tags: <document>,`
		`slaveDelay: <int>,`
		`votes: <number>`
		`}`

толькоАрбитр	Boolean значение	Необязательный. Применяется, только если значение <хост> является строкой. Если true, добавленный хост является арбитром.

```
rs.add('localhost:40001')
// or
rs.add('macbook.local:40001')
```

После чего также добавим арбитра.

```
rs.addArb(хост)
```

Добавляет нового арбитра в существующий набор реплик.

Параметры метода **addArb()** [47.] расшифровываются в Таблица 30.

Таблица 30. Параметры команды addArb()

Параметр	Тип	Описание

хост	Строка или документ	Обязательный. Новый элемент для добавления в набор реплик. Укажите либо в виде строки, либо в виде документа конфигурации. Задает имя хоста и, необязательно, номер порта члена-арбитра, добавляемого в набор реплик.
		{
		_id: <int>,
		host: <string>, // required
		arbiterOnly: <boolean>,
		buildIndexes: <boolean>,
		hidden: <boolean>,
		priority: <number>,
		tags: <document>,
		slaveDelay: <int>,
		votes: <number>
		}

```
rs.addArb('macbook.local:40002')
```

Арбитр не реплицирует данные, а выполняет роль наблюдателя. Как следует из названия, этот узел занимается арбитражем, в случае отказа первичного узла, арбитр помогает выбрать новый первичный узел.

Чтобы получить краткую сводку информации о репликах можно воспользоваться командой:

```
db.isMaster()
```

Получим вывод к консоль:

```
{
 "hosts" : [
  "macbook.local:40000",
  "macbook.local:40001"
 ],
 "arbiters" : [
  "macbook.local.40002:27017"
```

```
 ],
 "setName" : "myapp",
 "setVersion" : 3,
 "ismaster" : true,
 "secondary" : false,
 "primary" : "macbook.local:40000",
 "me" : "macbook.local:40000",
 "electionId" : ObjectId("7fffffff0000000000000001"),
 "lastWrite" : {
  "opTime" : {
   "ts" : Timestamp(1515887446, 1),
   "t" : NumberLong(1)
  },
  "lastWriteDate" : ISODate("2018-01-13T23:50:46Z")
 },
 "maxBsonObjectSize" : 16777216,
 "maxMessageSizeBytes" : 48000000,
 "maxWriteBatchSize" : 1000,
 "localTime" : ISODate("2018-01-13T23:50:55.010Z"),
 "maxWireVersion" : 5,
 "minWireVersion" : 0,
 "readOnly" : false,
 "ok" : 1
}
```

Более подробную информацию можно получить с помощью команды:

```
rs.status()
```

Мы настроили репликацию, теперь можно проверить работает ли она. Подключимся к первичному узлу и попробуем вставить в базу какой нибудь документ:

```
use books
db.users.insertOne({ nick: 'Merrick', status: 'frontend' })
```

Данные должно реплицироваться практически мгновенно, подключимся из другого окна терминала к вторичной базе:

```
mongo --port 40001
```

Драйвера для базы принимают список реплик к которым они могут подключатся и если одна из реплик недоступна, то драйвер подключится к следующей.

Чтобы разрешить считывание из вторичных узлов, нужно сообщить об этом к подключенному вторичному узлу, с помощью команды:

```
rs.slaveOk()
```

После чего выберем базу и посмотрим, что в ней:

```
use books
db.users.find() // Вернет только что добавленного юзера
```

Подробнее о методах репликации можно прочитать на официальном сайте MongoDB [48.].

2.14.3. Согласование реплик в БД NoSQL

Несмотря на разнообразие баз данных NoSQL, в процессе функционирования они выполняют некоторые общие функции, связанные с согласованием реплик [49.]:

- размещение реплик записей БД в кластере и обеспечение их согласования при обновлении записи,
- согласование версий реплики (сведение нескольких версий записи к одной записи в процессе ведения версий записи),
- согласование (восстановление) реплик после устранения сбоя в узле.

В базах данных NoSQL в основном используются два способа размещения и обновления реплик: по принципу «главный-подчиненный» (master-slave) и «по кольцу» (ring). Первый способ подразумевает хранение данных на master-узле и их репликацию на slave-узлы. Все изменения выполняются на master-узле, и они сохраняются в памяти этого узла. Slave-узлы периодически опрашивают masterузел, читают накопившиеся изменения и сохраняют их в своей памяти. Примерами таких баз данных являются MongoBD, HBase, Neo4j и др.

При размещении данных «по кольцу» необходимо определить, сколько в нем будет секций (v-узлов). Эти секции распределяются по серверам (по кольцу).

При включении записи в базу данных вычисляется хеш ключа, с помощью которого определяется номер секции (v-узла), по которому определяется сервер, где будет храниться эта запись. Другие реплики записи (их число равно N-1) сохраняются на следующих (N-1) серверах, расположенных по кольцу по часовой стрелке относительно первого сервера. Примерами баз данных с распределением по кольцу являются Riak, Amazon Dynamo и др.

Репликация используется для повышения производительности и отказоустойчивости. При обновлении записи остальные реплики обновляются не сразу, а в течение окна рассогласованности – времени, которое проходит от момента обновления записи на каком-либо узле до момента завершения обновлений копий записи на всех остальных узлах. Различают следующие виды согласованности:

- строгая согласованность –любой запрос на чтение всегда вернет последнее обновление записи;
- слабая согласованность – система не гарантирует, что запрос на чтение всегда вернет последнее обновление записи;
- согласованность в конечном счете – особая форма слабой согласованности: система хранения гарантирует, что, если впоследствии не будет никаких новых обновлений записи, в конечном счете запрос на чтение вернет последнее обновление.

Согласованность тесно связана с доступностью системы, а также с устойчивостью к потере связности. Эту связь изложил Эрик Брювер в так называемой теореме CAP. В теореме утверждается, что можно создать распределенную систему, которая будет:

1. согласованной (Consistent),
2. доступной (Available)
3. устойчивой к потере связности (Partition tolerant), но одновременно можно гарантировать только два из этих трех свойств. При согласованности в конечном счете гарантируются свойства 2 и 3.

Ведем следующие обозначения:

N – коэффициент репликации (количество реплик)

W – количество узлов, подтверждающих запись

R – количество узлов, с которых читают.

В одних базах данных NoSQL указанные выше параметры задаются явно, в других – не явно, т.е. в зависимости от режима репликации.

Явно задать параметры N, W, R непросто – это сложная задача, возникающая на этапе проектирования хранилища NoSQL.

2.14.4. Кворумы

Кворум представляет собой концепцию. Кластер имеет кворум при наличии и активности достаточного количества голосующих элементов кластера с одинаковым согласованным представлением кластера. Необходимое для кворума количество голосующих элементов кластера определяется режимом кворума.

После назначения голосов количество узлов, необходимых для кворума, определяется по формуле:

Общее число голосующих/2 + 1 = большинство (с округлением до меньшего)

Например, если в кластере пять узлов, то формула принимает вид

5/2 + 1 = 3,5

Дробный результат округляется в сторону уменьшения, поэтому в этом примере для кворума нужно три узла.

2.14.4.1. Кворум чтения

Со сколькими узлами следует установить контакт, чтобы гарантировать, что получаем самое последнее изменение (свежие данные).

Строгая согласованность: R + W > N.

2.14.4.2. Кворум записи

Сколько узлов должны подтвердить запись.

Строгая согласованность: W > N / 2. (см. Таблица 31)

Таблица 31. Параметры N, W, R согласования реплик для БД MongoDB

Название	Тип БД	Режим репликации	Механизм репликации	Значения N, W, R	Согласованность реплик
MongoDB	Документная	1.Replica sets (подмножество серверов, организованных как master-slave) 2.Master-slave	1.А)запись и чтение выполняется через primary-реплику Б)запись выполняется через primary-реплику, чтение через secondary-реплику	А)N=1 (primary-узел), W=1, R=1 Б) N= число серверов replica-sets, W=1, R=1 (это для каждой записи в primary-реплике; еще	А)строгая согласованность (R + W > N) Б) КС-согласован

Название	Тип БД	Режим репликации	Механизм репликации	Значения N, W, R	Согласованность реплик
			B)ReadConcern=”majority” WriteConcern=”majority” 2.Запрос на обновление выполняется только master-узлом, slave-узлы опрашивают главный узел; возможно чтение из slave-узлов	необходимо учитывать задержку на интервал опроса primary-узла) B) N= число серверов replica-sets, W и R вычисляются автоматически так, чтобы W+R>N. 2. N= число серверов в кластере, W=R=1 (это для каждой записи БД; еще необходимо учитывать задержку на интервал запроса master-узла.	ность (R + W <= N)

Подробнее про кворум отказоустойчивого кластера написано в статье Джона Сэвилла. [50.]

2.15. Организация кластера на основе MongoDB

Шардинг (фрагментация [51.]) – это подход к масштабируемости, когда отдельные части данных хранятся на разных серверах. Фрагментация решает проблему горизонтального масштабирования.

В динамично растущих системах, объемы данных, как правило, быстро увеличиваются и рано или поздно можно столкнуться с проблемой, когда текущих ресурсов машины будет не хватать для поддержания нормальной работы.

Для решения этой проблемы применяют масштабирование. Масштабирование бывает 2-х видов — горизонтальное и вертикальное.

Вертикальное масштабирование — это наращивание мощностей одной машины — добавление CPU, RAM, HDD.

Горизонтальное масштабирование — добавление новых машин к существующим и распределение данных между ними.

Первый случай наиболее простой, так как не требует дополнительных настроек приложения и какой-либо дополнительной конфигурации БД, но его недостаток в том, что мощности одной машины теоретически рано или поздно упрутся в тупик. Второй случай более сложен в конфигурации, но имеет следующие преимущества:

- Теоретически бесконечное масштабирование (машин можно поставить сколько угодно, вопрос упирается только в денежные издержки)
- Бóльшая безопасность данных (только при использовании репликации) — машины могут располагаться в разных дата центрах (при падении одной из них, останутся другие)

Фрагментация является частным случаем горизонтального масштабирования. Суть ее в разделении (партиционировании) базы данных на отдельные части по определенному правилу так, чтобы каждую из них можно было вынести на отдельный сервер. Однако реплицирование данных является обязательным требованием для шардирования в Mongo. Сервер конфигурации и каждый шард представляют из себя так называемый Реплика-сет.

Это группа экземпляров mongod (демон mongoDB), хранящих одинаковые наборы данных. В наборе копии один узел – ключевой, который получает все операции записи. Все остальные узлы – вторичны, принимают операции из первого, таким образом, храня такие же записи, как и первичный узел. Набор копий может иметь только один первичный узел. В случае его отказа другой узел может занять его место либо случайным образом (схема без арбитража), либо по решению сервера-арбитра.

Свойства наборы копий (Replica Set):

- Любой узел может быть первичным
- Все операции записи идут в первичный узел
- Кластер состоит из N узлов
- Автоматическая отказоустойчивость
- Автоматическое восстановление данных
- Автоматический выбор первичного ключа

Все операции записи, удаления, обновления, попадают в мастер (primary), а затем записываются в специальную коллекцию oplog, откуда асинхронно попадают на реплики — repl.1 и repl.2 (secondary). Таким образом происходит дублирование данных. Такой подход имеет ряд преимуществ:

- Избыточность обеспечивает безопасность данных — при падении мастера, происходит голосование между репликами и одна из них становится мастером
- Мастер и реплики могут располагаться в разных дата центрах — это может оказаться полезным, если сервер повреждается физически (пожар в дата центре)
- Реплики могут использоваться для более эффективного чтения данных.

Например, есть приложение, которое имеет клиентскую аудиторию в Европе и в США. Одну из реплик можно поместить на территории США и настроить так, чтобы клиенты из США вычитывали данные именно из нее.

Непосредственно фрагментация происходит следующим образом — данные попадают в фрагменты документа одинакового размера (далее порции) по определенному диапазону заданного поля — shard key. Сначала создается всего одна порция и диапазон значений, которые она принимает, лежит в пределах (-∞, +∞). Когда размер этой порции достигает chunksize, mongos оценивает значение всех ключей внутри порции и делит порцию таким образом, чтобы данные были разделены примерно поровну. Для примера предположим, что у нас есть 4 документа с полями name, age и id. id — является shard key:

```
{“name”: “Max”, “age”: 23, “id”: 23}
{“name”: “John”, “age”: 28, “id”: 15}
{“name”: “Nick”, “age”: 19, “id”:56}
{“name”: “Carl”, “age”: 19, “id”: 78}
```

Предположим размер chunksize, уже достигнут. В данном случае Mongos разделит диапазон примерно так (-беск,45]; (45, +беск). Получим 2 порции: (см. Рисунок 68)

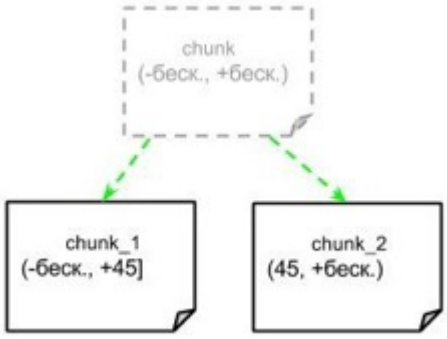


Рисунок 68. Разделение порции данных

При появлении новых документов, они будут записываться в порцию, которая соответствует диапазону shardKey. По достижению chunksize (предельное количество записей в одну порцию) разделение произойдет снова и диапазон будет еще уже и так далее. Все порции хранятся в фрагментах.

Важно заметить, что при достижении какой-либо порции неделимого диапазона, например (44, 45], деление происходить не будет и порция будет расти свыше chunksize. Поэтому следует внимательно выбирать shard key, чтобы это была как можно наиболее случайная величина. Например, если нам надо заполнить БД всеми людьми на планете, то удачными выборами shard key были бы номер телефона, идентификационный налоговый номер, почтовый индекс. Неудачными — имя, город проживания.

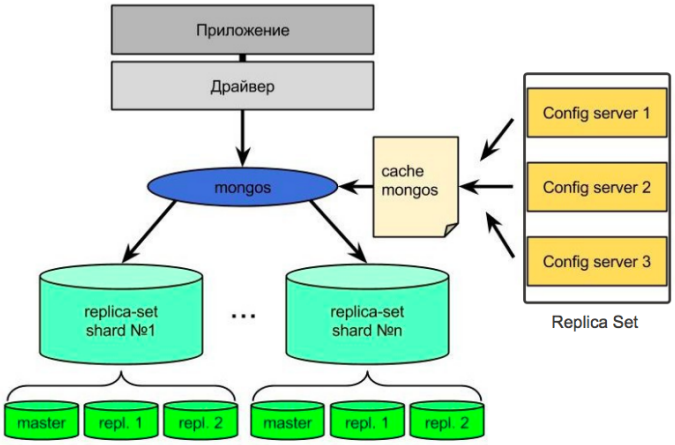


Рисунок 69. Схема взаимодействия процессов в MongoDB.

На схеме (см. Рисунок 69) справа показан config сервер, его отличие от mongod в том, что он не обрабатывает клиентские запросы, а является хранилищем метаданных — он знает физические адреса всех порций данных, знает какую порцию, на каком фрагменте искать и какой диапазон у того или иного фрагмента. Все эти данные он хранит в специально отведенном месте —config database.

В данной схеме также присутствует роутер запросов — mongos, на него возлагаются следующие задачи:

- Кэширование данных, хранимых на config сервере
- Роутинг запросов чтения и записи от драйвера — маршрутизация запросов от приложений на нужные фрагменты, mongos точно знает где физически находится та или иная порция данных
- Запуск фонового процесса “Балансер”

Функция балансера заключается в миграции порций из одного фрагмента на другой. Процесс происходит примерно так: балансер отсылает команду moveChunk на фрагмент, из которого будет мигрировать chunk (порция), фрагмент получая эту команду, запускает процесс копирования порции данных на другой фрагмент. После того, как все документы скопированы, происходит синхронизация документов между этими 2-мя порциями, т.к. пока происходила миграция, в исходную порцию данных могли добавиться новые данные. После окончания синхронизации фрагмент, который принял новую порцию, отправляет ее адрес config серверу, чтобы тот, в свою очередь, обновил его в кэше mongos. По окончанию этого процесса, если на исходной порции нет открытых курсоров, он удаляется. 3 config сервера обеспечивают избыточность данных, и если упадет один, mongos все равно будет получать актуальные адреса порций данных от других config серверов.

2.15.1. Настройка фрагментации для БД

2.15.1.1. Подготовка директорий

Перед началом работы необходимо запастись директориями, в которых будут располагаться данные и служебные файлы (журналы и прочее) каждого сервера и самой БД. В данной работе рассматривается пример с двумя фрагментами. Так как Mongo требует реплика-сеты каждого фрагмента и конфигурационные сервера минимум из трех участников, то потребуются:

- Папка фрагмента «a», в которой будут папки «a0», «a1», «a2»
- Папка фрагмента «b», в которой будут папки «b0» «b1» «b2»
- Папка для работы самой БД — db, в которой будут папки cfg0, cfg1, cfg2, соответствующие набору реплик конфигурационного сервера.

Все эти директории создаются в корневом каталоге системы в папке /data с помощью команды sudo mkdir. Важно заметить, что в этой директории нельзя создавать папки если вы не root, однако ядро безопасности MongoDB требует, чтобы директории в которых она работает принадлежали пользователю, запустившему её. Поэтому после создания всех папок нужно сделать основного пользователя их владельцем следующей командой:

```
$ sudo chown parallels * -R /data
```

Эта команда расшифровывается так— рекурсивно изменить владельца всех файлов на пользователя parallels в папке data и всех вложенных в неё.

Эта команда расшифровывается так— рекурсивно изменить владельца всех файлов на пользователя parallels в папке data и всех вложенных в неё.

2.15.1.2. Создание наборов реплик

Последовательно создадим наборы реплик сначала конфигурационных серверов, а затем и серверов для фрагментов. Каждый сервер— это запущенный процесс mongod. Для того чтобы не загромождать систему, мы будем запускать их в фоновом режиме (если потребуется их остановка, это всегда можно сделать через саму mongo). Для поднятия каждого сервера нужно указать набор параметров.

2.15.1.3. Параметры запуска mongod

Параметры запуска mongod представлены в Таблица 32.

Таблица 32. Параметры запуска mongod

Параметр	Пояснение
--dbpath	Путь к файлам этого сервера
--logpath	Путь к файлу логов этого сервера
--port	Указание порта, через который он будет слушать соединения
--fork	Флаг, означающий что mongod должен быть поднят в фоновом режиме (при запуске без этого параметра каждая служба заберет одно окно терминала в свое пользование, что затрудняет работу пользователя)
--configsvr	Флаг, означающий что данный сервер исполняет роль конфигурационного
--shardsvr	Флаг, означающий что данный сервер исполняет роль элемента фрагмента
--replSet	Указание к какому набору копий принадлежит сервер. В этой работе a и b для соответствующих фрагментов и «c» для конфигурационных серверов
--smallfiles	Флаг упрощения логов, для сохранения места на диске
--oplog	Размер служебной коллекции oplog используемой при репликации данных

Для конфигурационных серверов выбраны порты: 26050, 26051, 26052 Для серверов фрагментов выбраны порты:

- Фрагмент А: 27000, 27001, 27002
- Фрагмент Б:27003, 27004, 27005

  Для службы mongos выбран порт 26060

```
      //Запуск config-mongod
      $ mongod --configsvr --dbpath /data/db/cfg0 --port 26050 --logpath
/data/db/log.cfg0 --fork --replSet c

      $ mongod --configsvr --dbpath /data/db/cfg1 --port 26051 --logpath
/data/db/log.cfg1 --fork --replSet c

      $ mongod --configsvr --dbpath /data/db/cfg2 --port 26052 --logpath
/data/db/log.cfg2 --fork --replSet c

      //Запуск серверов для фрагментов
      $mongod --shardsvr --replSet a --dbpath /data/a/a0 --logpath
/data/a/log.a0 --port 27000 --fork --logappend --smallfiles --oplogsize 50

      $mongod --shardsvr --replSet a --dbpath /data/a/a1 --logpath
/data/a/log.a1 --port 27001 --fork --logappend --smallfiles --oplogsize 50

      $mongod --shardsvr --replSet a --dbpath /data/a/a2 --logpath
/data/a/log.a2 --port 27002 --fork --logappend --smallfiles --oplogsize 50

      $mongod --shardsvr --replSet b --dbpath /data/b/b0 --logpath
/data/a/log.b0 --port 27003 --fork --logappend --smallfiles --oplogsize 50

      $mongod --shardsvr --replSet b --dbpath /data/b/b1 --logpath
/data/a/log.b1 --port 27004 --fork --logappend --smallfiles --oplogsize 50

      $mongod --shardsvr --replSet b --dbpath /data/b/b2 --logpath
/data/a/log.b2 --port 27005 --fork --logappend --smallfiles --oplogsize 50
```

Все необходимые mongod-процессы запущены, однако они ничего не знают друг о друге. Их нужно соединить с помощью mongos. Для этого прописываем в терминал следующую команду:

```
      $ mongos --configdb localhost:26050,localhost:26051,localhost:26052
--fork --logappend --logpath /data/db/log.mongos --port 26060
```

Некоторые комментарии по данной команде. Флаг --configdb означает, что перечисленные далее адреса принадлежат конфигурационным серверам, которые работают вместе в рамках одного набора копий. Эту команду очень важно ввести аккуратно из-за её большого размера— даже 1 лишний пробел приведет к ошибке парсинга строки. После запятых, несмотря на общее правило, здесь не ставятся пробелы.

Теперь mongos знает откуда брать служебную информацию, но ничего не знает о том, где хранятся сами данные. Для следующего этапа, нужно вызвать shell самой БД.

```
$ mongo --port 27000
```

Устанавливается соединение с первым из серверов фрагмента A. Появляется возможность управления им из данной оболочки. Следующая последовательность команд инициирует набор копий (до этого существовавший лишь формально, процессы mongod были просто помечены как принадлежащие одной группе) и связывает сервера друг с другом.

Будем использовать метод **initiate()**. Он инициирует набор реплик. Метод может принимать аргумент в форме документа, который содержит конфигурацию набора реплик.

```
rs.initiate(<настройки>)
```

Параметры метода **initiate()** расшифровываются в Таблица 33.

Таблица 33. Параметры команды initiate()

Параметр	Тип	Описание
Настройки	Документ	Необязательный. Документ, который определяет конфигурацию для нового набора реплик. Если конфигурация не указана, MongoDB использует конфигурацию набора реплик по умолчанию.

Объект настроек содержит поля: (см. Таблица 34)

Таблица 34. Поля объекта настроек

Свойство	Тип	Описание
_id	string	Установлен на имя набора реплик, указанное либо в replication.replSetName, либо в параметре --replSet.

Свойство	Тип	Описание
configsvr	boolean	Указывает, используется ли набор реплик для серверов конфигурации фрагментированного кластера. Установите значение true, если набор реплик предназначен для серверов конфигурации. По умолчанию false.
members	array	Содержит документ для каждого члена набора реплик.

Будем использовать метод **add()**. Добавляет участника в набор реплик. Для запуска метода необходимо подключиться к первичному набору реплик.

Параметры метода **add ()** расшифровываются в Таблица 35.

Таблица 35. Параметры команды add()

Параметр	Тип	Описание
host	Документ или строка	Новый член для добавления в набор реплик. Укажите либо в виде строки, либо в документе конфигурации: Если это документ, укажите документ конфигурации элемента набора реплик, найденный в массиве members. Следует указать поле хоста в документе конфигурации участника. `{ _id: <int>, host: <string>, arbiterOnly: <boolean>, buildIndexes: <boolean>, hidden: <boolean>, priority: <number>, tags: <document>, slaveDelay: <int>, votes: <number> }` Если параметр указывается как строка, укажите имя хоста и, опционально, номер порта для нового участника.
arbiterOnly	boolean	Необязательный. Применяется, только если значение <host> является строкой. Если true, добавленный хост является арбитром.

```
>rs.initiate()
>rs.add(“127.0.0.1:27001”)
>rs.add(“127.0.0.1:27002”)
>exit
```

Аналогичную процедуру нужно повторить и для фрагмента Б.

```
$mongo --port 27003
>rs.initiate()
>rs.add(“127.0.0.1:27004”)
>rs.add(“127.0.0.1:27005”)
>exit
```

2.15.2. Фрагментация целевой БД

Работа с БД начинается с подключения оболочки mongoDB к mongos. Он расположен на порту 26060.

Будем использовать метод **addshard()**.Добавляет набор фрагментов в кластер. Этот метод должен быть запущен на экземпляре Mongos.

```
sh.addShard(<url>)
```

Параметры метода **addshard ()** расшифровываются в Таблица 36.

Таблица 36. Параметры команды addShard()

Параметр	Тип	Описание
host	string	Обязательный. Имя набора реплик, имя хоста и порт как минимум одного члена набора реплик фрагмента. Любые дополнительные имена хостов членов набора реплик должны быть разделены запятыми.

Будем использовать метод **enableSharding()**. В оболочке mongo, подключенной к mongos, используйте метод **sh.enableSharding()** для применения фрагментации в целевой базе данных. После применения фрагментации для базы данных вы можете использовать **sh.shardCollection ()** для фрагментации коллекций в этой базе данных.

```
sh.enableSharding(<database>, <primaryShard>)
```

Параметры метода **enableSharding ()** расшифровываются в Таблица 37.

Таблица 37. Параметры команды enableSharding()

Параметр	Тип	Описание
database	String	Обязательный. База данных, для которой вы хотите применить фрагментацию.
primaryShard	String	Необязательный. Основной фрагмент для базы данных(где хранятся все данные БД). Вместо того, чтобы явно указывать основной фрагмент, рекомендуется позволить балансировщику выбрать основной фрагмент.

Будем использовать метод **shardCollection()**. Фрагментирует коллекции с использованием ключа в качестве ключа фрагмента.

```
sh.shardCollection(namespace, key, unique, options)
```

Параметры метода **shardCollection ()** расшифровываются в Таблица 38.

Таблица 38. Параметры команды shardCollection()

Параметр	Тип	Описание
namespace	String	Пространство имен коллекции, которую нужно фрагментировать, в форме <база данных>. <коллекция>.
key	document	Документ спецификации индекса для использования в качестве ключа фрагмента. Ключ фрагмента определяет, как MongoDB распределяет документы между фрагментами. Значение документа должно быть одним из следующих: 1 для ранжированного фрагментирования, "hashed", чтобы указать хешированный ключ фрагмента.
unique	boolean	Необязательный. Когда true, опция unique гарантирует, что базовый индекс применяет ограничение уникальности. По умолчанию false.
options	document	Необязательный. Документ с параметрами, содержащий необязательные поля, включая numInitialChunks и параметры сравнения.

Объект параметров содержит следующие свойства: (см. Таблица 39)

Таблица 39. Свойства объекта с параметрами

Свойство	Тип	Описание
numInitialChunks	integer	Необязательный. Определяет количество порций, создаваемых изначально при разбиении пустой коллекции с помощью хешированного ключа фрагмента. Затем MongoDB создаст и уравновесит фрагменты по всему кластеру. Значение numInitialChunks должно быть меньше 8192 на сегмент.
collation	document	Необязательный. Если для коллекции, указанной

Свойство	Тип	Описание
		для shardCollection, используется сравнение по умолчанию, необходимо включить документ с настройками сравнения с {locale: "simple"}, иначе команда shardCollection завершится неудачно.

```
$mongo --port 26060
>sh.status() //проверка статуса БД
>sh.addshard("a/localhost:27000")//добавление фрагмента А к mongos
>sh.addshard("b/localhost:27003")//добавление фрагмента Б к mongos

//Выключаем балансер нагрузки
>sh.stopBalancer()

//Создаем фрагментируемую базу данных «shardTestDB»
>use shardTestDB
>sh.enableSharding("shardTestDB")
```

```
//Создаем коллекцию users, в которой ключом для фрагментации станут значения поля username
>sh.shardCollection("shardTestDB.users", {"username": 1})

//Перейдем в служебную БД config, встроенную в MongoDB, используемую для хранения общих конфигурационных настроек. Изменим размер порции данных с 64 Мб до 1 Мб

>use config
>db.settings.find({"_id": "chunksize"})
>db.settings.save({"_id": "chunksize", value: 1})

//вернемся в исходную БД
>use shardTestDB
```

Создается 100000 записей в коллекции users с использованием цикла for (для генерации числовых идентификаторов записей) и команды db.insert(): (см. Рисунок 70)

```
for(var i=0; i<100000; i++){
    db.users.insert({"username": "user"+i, "created at" : new Date()})
    }
//Вывод состояния фрагментов после заполнения на экран
>sh.status()
```

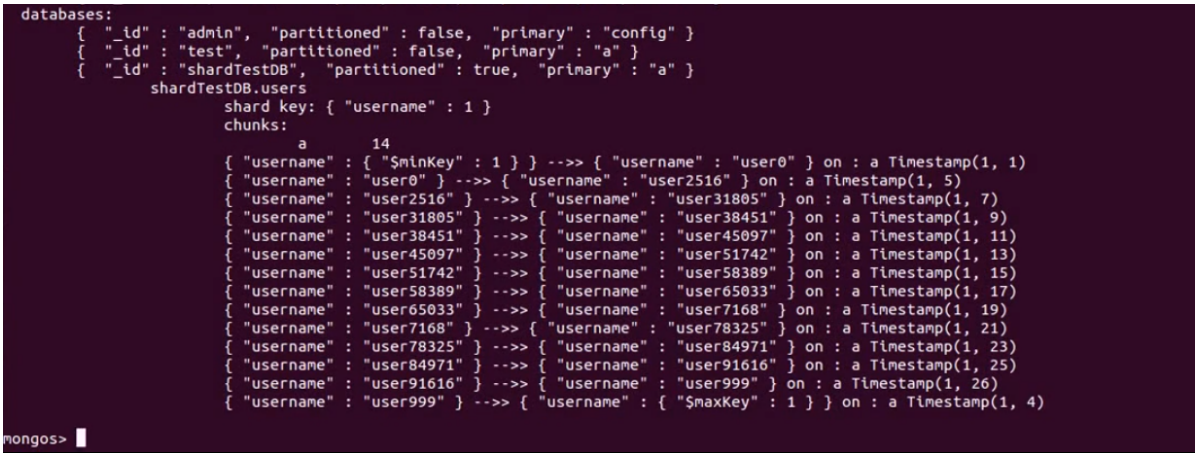


Рисунок 70. Первичное распределение данных по фрагментам.

100000 записей уместились в 14 порций размером 1 Мб. В начальный момент времени вся информация на фрагменте А. Теперь активируется балансер нагрузки, ответственный за перераспределение данных между фрагментами.

```
>sh.startBalancer()

>sh.status() //Вывод состояния фрагментов на экран
```

Балансер нагрузки постепенно перераспределяет данные между фрагментами. Периодически вводя команду **sh.status()** можно видеть изменения в количестве порций в фрагментах. На фрагмент Б перемещаем 2 порции данных (см. Рисунок 71).

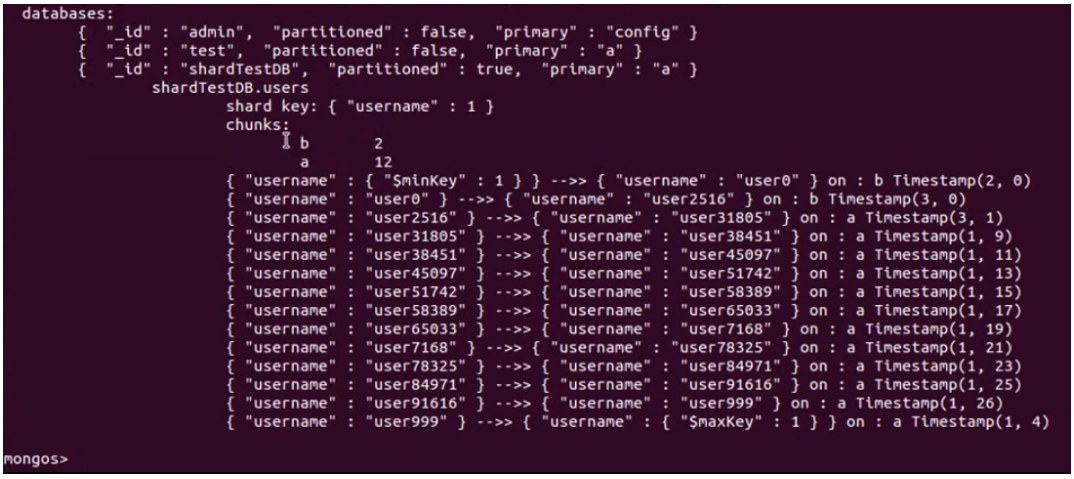


Рисунок 71. На фрагмент Б перемещено 2 порции данных.

Этот процесс продолжается до тех пор, пока распределение не станет равномерным. Конечный результат представлен на скриншоте ниже (см. Рисунок 72).



Рисунок 72. Конечный результат.

В настоящее время MongoDB получила широкое распространение и используется во множестве проектов. Она поддерживает множество функций, сложных агрегаций и возможностей манипулирования данными. Является классической документоориентированной СУБД, не имеет четкой схемы данных. В последнее время MongoDB развивается в сторону HiLoad и BigData проектов, в частности не так давно появился коннектор для Spark. MongoDB хорошо подойдет как для быстрого старта MVP-проекта, так и для сложных BigData решений.

3. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Каковы отличительные особенности БД NoSql? Каких типов они бывают?
2. Что такое документная БД? Какую модель данных она реализует?
3. Какие возможности предоставляют документные БД?
4. Как добавлять и изменять объекты БД MongoDb?
5. Как добавлять в объекты БД MongoDb коллекции и вложенные структуры?
6. Какие типы запросов поддерживают документные БД?
7. Какова структура запроса к данным в БД MongoDb?
8. Как выполнить запросы с фильтрацией и проекцией?
9. Как выполнить запросы с сортировкой, группировкой и агрегированием?
10. Как можно задать связь между объектами и выполнить переход по связи?

4. СПИСОК ИСТОЧНИКОВ

1. Фаулер, Мартин, Садаладж, Прамодкумар Дж. NoSQL: новая методология разработки нереляционных баз данных. : Пер. с англ. - М.: ООО "И.Д. Вильямс", 2013г.
2. Что такое документная база данных? – Текст. Изображение : электронные // Сервисы облачных вычислений – Amazon Web Services (AWS): [сайт]. – URL: https://aws.amazon.com/ru/nosql/document/ (дата обращения: 11.05.2022)
3. 11 типов современных баз данных: краткие описания, схемы и примеры БД. – Текст. Изображение : электронные // Библиотека программиста: [сайт]. – URL: https://proglib.io/index.php/p/11-tipov-sovremennyh-baz-dannyh-kratkie-opisaniya-shemy-i-primery-bd-2020-01-07 (дата обращения: 11.05.2022)
4. MongoDB | Устройство базы данных. Документы. – Текст. Изображение : электронные // METANIT.COM Сайт о программировании: [сайт]. – URL: https://metanit.com/nosql/mongodb/2.1.php (дата обращения: 25.04.2022).
5. MongoDB | Введение. – Текст. Изображение : электронные // METANIT.COM Сайт о программировании: [сайт]. – URL: https://metanit.com/nosql/mongodb/1.1.php (дата обращения: 11.05.2022)
6. Ответы на вопросы на собеседование MongoDB. | Вопросы на собеседование Junior Java Developer. – Текст. Изображение : электронные // Вопросы на собеседование Junior Java Developer: [сайт]. – URL: https://jsehelper.blogspot.ru/2016/05/mongodb.html (дата обращения: 11.05.2022)
7. MongoDB – Краткое руководство. – Текст. Изображение : электронные // Уроки и статьи по программированию и IT: [сайт]. – URL: https://coderlessons.com/tutorials/bazy-dannykh/uchitsia-mongodb/mongodb-kratkoe-rukovodstvo (дата обращения: 11.05.2022)
8. Introduction to MongoDB – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/introduction/ (дата обращения: 11.05.2022)
9. Data Modeling Introduction – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/core/data-modeling-introduction/ (дата обращения: 11.05.2022)

10. Официальный сайт MongoDB. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://www.mongodb.com/ (дата обращения: 25.04.2022).
11. How to get started with MongoDB in 10 minutes. – Текст. Изображение : электронные // freeCodeCamp Programming Tutorials: Python, JavaScript, Git & More: [сайт]. – URL: https://www.freecodecamp.org/news/learn-mongodb-a4ce205e7739/ (дата обращения: 11.05.2022)
12. Примеры основных запросов MongoDB. – Текст. Изображение : электронные // Веб-хостинг PHP, MYSQL: [сайт]. – URL: http://hostciti.net/support/programmistam/mongodb/136.html (дата обращения: 11.05.2022)
13. MongoDB – Создать базу данных. – Текст. Изображение : электронные // Уроки и статьи по программированию и IT: [сайт]. – URL: https://coderlessons.com/tutorials/bazy-dannykh/uchitsia-mongodb/mongodb-sozdat-bazu-dannykh (дата обращения: 11.05.2022)
14. Руководство по MongoDB. Создание коллекций. – Текст. Изображение : электронные // PROSELYTE – Software Engineering: [сайт]. – URL: https://proselyte.net/tutorials/mongodb/create_collection/ (дата обращения: 11.05.2022)
15. db.collection.insert() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.insert/index.html (дата обращения: 11.05.2022)
16. db.collection.save() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.save/index.html (дата обращения: 11.05.2022)
17. Руководство по MongoDB. Вставка документа. – Текст. Изображение : электронные // PROSELYTE – Software Engineering: [сайт]. – URL: https://proselyte.net/tutorials/mongodb/insert_document/ (дата обращения: 11.05.2022)
18. MongoDB: Запросы. – Текст. Изображение : электронные // Habr.: [сайт]. – URL: https://habr.com/ru/post/134590/ (дата обращения: 11.05.2022)

19. db.collection.find() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.find/index.html (дата обращения: 11.05.2022)
20. Query and Projection Operators – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/operator/query/ (дата обращения: 11.05.2022)
21. MongoDB | Выборка из БД. – Текст. Изображение : электронные // METANIT.COM Сайт о программировании: [сайт]. – URL: https://metanit.com/nosql/mongodb/2.4.php (дата обращения: 11.05.2022)
22. db.collection.findOne() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.findOne/index.html (дата обращения: 11.05.2022)
23. db.collection.update() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.update/index.html (дата обращения: 11.05.2022)
24. Update Operators – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/operator/update/ (дата обращения: 11.05.2022)
25. db.collection.updateMany() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.updateMany/index.html (дата обращения: 11.05.2022)
26. db.collection.remove() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.remove/index.html (дата обращения: 11.05.2022)

27. cursor.sort() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/cursor.sort/index.html (дата обращения: 11.05.2022)

28. db.collection.count() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.count/index.html (дата обращения: 11.05.2022)

29. db.collection.distinct() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.distinct/index.html (дата обращения: 11.05.2022)

30. cursor.limit() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/cursor.limit/index.html (дата обращения: 11.05.2022)

31. MongoDB | Установка ссылок в БД. – Текст. Изображение : электронные // METANIT.COM Сайт о программировании: [сайт]. – URL: https://metanit.com/nosql/mongodb/2.11.php (дата обращения: 11.05.2022)

32. Aggregation Operations – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/aggregation/ (дата обращения: 11.05.2022)

33. db.collection.aggregate() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.aggregate/index.html (дата обращения: 11.05.2022)

34. Aggregation Pipeline Stages – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/operator/aggregation-pipeline/ (дата обращения: 11.05.2022)

35. $group (aggregation) – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL:

https://docs.mongodb.com/manual/reference/operator/aggregation/group/#accumulators-group (дата обращения: 11.05.2022)

36. MongoDB | Работа с индексами. – Текст. Изображение : электронные // METANIT.COM Сайт о программировании: [сайт]. – URL: https://metanit.com/nosql/mongodb/2.12.php (дата обращения: 11.05.2022)
37. db.collection.createIndex() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.createIndex/index.html (дата обращения: 11.05.2022)
38. db.collection.explain() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.explain/index.html (дата обращения: 11.05.2022)
39. db.collection.dropIndex() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/db.collection.dropIndex/index.html (дата обращения: 11.05.2022)
40. Кластер (группа компьютеров). – Текст. Изображение : электронные // Википедия – свободная энциклопедия: [сайт]. – URL: https://ru.wikipedia.org/wiki/%D0%9A%D0%BB%D0%B0%D1%81%D1%82%D0%B5%D1%80_(%D0%B3%D1%80%D1%83%D0%BF%D0%BF%D0%B0_%D0%BA%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80%D0%BE%D0%B2) (дата обращения: 11.05.2022)
41. Руководство по MongoDB. Репликация. – Текст. Изображение : электронные // PROSELYTE – Software Engineering: [сайт]. – URL: http://proselyte.net/tutorials/mongodb/replication/ (дата обращения: 11.05.2022)
42. Руководство по MongoDB. Репликация. - PROSELYTE. – Текст. Изображение : электронные // PROSELYTE – Software Engineering: [сайт]. – URL: https://proselyte.net/tutorials/mongodb/replication/ (дата обращения: 11.05.2022)
43. Репликация MongoDB. – Текст. Изображение : электронные // Alex Kurapov: [сайт]. – URL: http://alexkurapov.blogspot.com/2018/10/mongodb_15.html (дата обращения: 11.05.2022)

44. Replication – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/replication/ (дата обращения: 11.05.2022)
45. rs.initiate() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/rs.initiate/index.html (дата обращения: 11.05.2022)
46. rs.add() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/rs.add/index.html (дата обращения: 11.05.2022)
47. rs.addArb() – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/rs.addArb/index.html (дата обращения: 11.05.2022)
48. Replication Methods – MongoDB Manual. – Текст. Изображение : электронные // MongoDB: The Application Data Platform | MongoDB: [сайт]. – URL: https://docs.mongodb.com/manual/reference/method/js-replication/ (дата обращения: 11.05.2022)
49. Цвященко Е.В. Модели процессов согласования реплик в базах данных NoSQL. – Текст. Изображение : электронные // Национальный исследовательский университет «МЭИ» : [сайт]. – URL: https://mpei.ru/diss/Lists/FilesDissertations/193-%D0%94%D0%B8%D1%81%D1%81%D0%B5%D1%80%D1%82%D0%B0%D1%86%D0%B8%D1%8F.pdf (дата обращения: 11.05.2022)
50. Что представляет собой кворум отказоустойчивого кластера Windows Server 2008? – Текст. Изображение : электронные // Издательство «Открытые системы»: [сайт]. – URL: https://www.osp.ru/winitpro/2008/03/5270403/ (дата обращения: 11.05.2022)
51. MongoDB Sharded Cluster. – Текст. Изображение : электронные // Национальная библиотека им. Н.Э. Баумана: [сайт]. – URL: https://ru.bmstu.wiki/MongoDB_Sharded_Cluster (дата обращения: 11.05.2022)